

Decision Quotient: A Regime-Sensitive Complexity Theory of Exact Relevance Certification

Tristan Simas
McGill University
`tristan.simas@mail.mcgill.ca`

March 10, 2026

Abstract

How much of the input must you keep to make the same decision? We show this seemingly simple question has radically different complexity depending on whether you are reasoning statically, stochastically, or sequentially. We study exact relevance certification for coordinate-structured decision problems and show that it admits a regime-sensitive complexity theory. For a decision problem $\mathcal{D} = (A, S, U)$, this yields the decision problems SUFFICIENCY-CHECK, MINIMUM-SUFFICIENT-SET, and ANCHOR-SUFFICIENCY, together with the optimizer quotient as the coarsest abstraction that preserves optimal-action distinctions.

The central result is a sharp complexity escalation across regimes. Surprisingly, the search for a minimum sufficient set in the static regime collapses to relevance containment, placing it in coNP rather than the expected Σ_2^P . In contrast, stochastic and sequential formulations are harder: stochastic sufficiency is PP -complete, stochastic anchor and minimum queries sit at the existential counting boundary with PP -hardness and paper-level NP^{PP} upper bounds, and sequential sufficiency, minimum, and anchor queries are PSPACE -complete. Under standard assumptions, this yields a strict hierarchy: $\text{static} \subset \text{stochastic} \subset \text{sequential}$.

We prove an encoding-sensitive dichotomy where the tractable-hard boundary is sharp, and we identify an integrity-competence trilemma: no exact certifier can simultaneously be sound, complete, and polynomial-budgeted on the hard regime. We also isolate six structural conditions under which exact certification remains polynomial. This establishes both tractability islands and fundamental limits for exact simplification.

The Lean 4 artifact certifies the core reduction-correctness lemmas, size-bounded hardness packages, exact finite deciders, explicit-state step-counting upper-bound wrappers, and the scoped witness-over-verifier and internal completeness-style layers used to support the stochastic existential classification, ensuring the finite reduction core is mechanically verified.

1 Introduction

Which coordinates of a decision problem can be hidden without changing the optimal action, and how hard is it to certify that claim exactly? We expected one answer. We found three fundamentally different ones. We study this exact-certification question across static, stochastic, and sequential

regimes. For a decision problem $\mathcal{D} = (A, S, U)$ with $S = X_1 \times \cdots \times X_n$, a coordinate set I is sufficient when agreement on I forces agreement on the optimal-action set:

$$s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s').$$

This is the basic problem of exact relevance certification: how much of the state must remain visible to preserve the decision boundary, and what it takes to certify that the rest is irrelevant. The question resembles subset selection, but exact certification has a different quantifier structure and, as the paper shows, a different complexity landscape.

This is a certification problem rather than a forward-evaluation problem. Evaluating $U(a, s)$ or $\text{Opt}(s)$ on a fully specified state is one task; proving that a family of coordinates is irrelevant requires ruling out every counterexample pair of states. That universal structure drives the static regime. A first structural point is worth stating up front: MINIMUM-SUFFICIENT-SET looks existential-universal on its face, but in the static regime it collapses because a set is sufficient exactly when it contains every relevant coordinate. So the minimum sufficient set is just the relevant-coordinate set. At the same time, the optimizer quotient is not an arbitrary definition: it is the canonical coarsest abstraction that preserves optimal-action distinctions. ANCHOR-SUFFICIENCY does not collapse in the same way, because the anchor assignment is itself part of the existential choice and must then satisfy a universal fiberwise condition. At this static boundary the paper touches classical reduct-style attribute reduction, but its main object is broader: the same exact certification predicate is then transported to stochastic conditioning and sequential policy preservation.

This distinction also explains why exactness matters. When a system replaces exact relevance certification with a heuristic simplification, the omitted relevance does not disappear; it is shifted into residual uncertainty that must be handled elsewhere in the system. We refer to that externalized burden as the *simplicity tax*. The results of this paper show that this tax is not merely rhetorical: exact simplification can be computationally expensive, but inexact simplification does not erase decision-relevant structure; it relocates it.

Informal interpretive guide. The formal objects studied here admit a compact intuitive reading. A relevant coordinate is a point of *leverage*: changing it can change the decision. The optimizer quotient isolates the decision *signal*: it is the coarsest abstraction that still preserves optimal-action distinctions. The static collapse expresses *inevitability*: in the static regime, the optimal simplification is forced by the structure of relevance rather than discovered by search. The simplicity tax is *liability*: omitted exact relevance does not disappear, but must be handled elsewhere in the system. At the task level, sufficiency is *observation*, anchor sufficiency is *discovery*, and sequential certification is *exposure*: omitted information may appear harmless now while still creating future liability elsewhere in the system.

The paper’s central claim is that this is not a collection of isolated hardness facts. Once the same exact-certification question is tracked across richer input models, the difficulty escalates in a systematic way. Static certification is governed by counterexample exclusion, stochastic certification by majority-style conditional comparisons, and sequential certification by policy preservation across evolving information states. Read as a complexity matrix, adding probability lifts exact certification from coNP to PP, and adding time lifts it again to PSPACE. The optimizer quotient packages the same issue structurally: it is the coarsest abstraction of the state space that preserves optimal-action distinctions.

Throughout, we work with finite coordinate-structured decision problems under explicit encoding regimes. Section 2.4 fixes the computational model used by every complexity claim.

1.1 Contributions

The paper is best read as one theorem package with four linked takeaways: a structural static collapse, a regime-sensitive complexity matrix, a sharp phase transition between tractable and hard exact certification, and a trilemma for exact certifiers in the hard regime.

1. **The Static Collapse Theorem.** MINIMUM-SUFFICIENT-SET appears to require Σ_2^P search: existentially choose a set, universally verify sufficiency. We show it is coNP-complete. The existential layer collapses because sufficient sets are exactly the supersets of the relevant-coordinate set. By contrast, ANCHOR-SUFFICIENCY remains Σ_2^P -complete because its existential anchor choice does not collapse.
2. **Regime Hierarchy.** We track the same exact-certification question across static, stochastic, and sequential regimes. The complexity escalates from coNP and Σ_2^P in the static regime, to PP for stochastic sufficiency and an existential counting boundary for stochastic anchor/minimum, to PSPACE for the sequential sufficiency, anchor, and minimum queries. The stochastic anchor and minimum variants are PP-hard and admit paper-level NP^{PP} upper bounds, so the paper-level landscape is complete even though the stochastic existential cells remain stated as gap-bounded rather than collapsed to PP.
3. **Sharp phase transition between tractable and hard exact certification.** We prove an explicit-state versus succinct dichotomy: logarithmic-size sufficient sets admit polynomial-time algorithms in the explicit regime, while linear-size sufficient sets in the succinct regime inherit ETH-conditioned exponential lower bounds. We also isolate six structured tractable subcases under standard restrictions.
4. **Impossibility Theorem.** In the hard regime, no exact certifier can be simultaneously sound, complete, and polynomial-budgeted. One of these must fail: abstain on some instances, weaken guarantees, or accept incorrect outputs. The same section also isolates witness-checking lower bounds, approximation obstructions, and the externalization of omitted exact relevance.
5. **Lean-certified reduction and finite-decision core.** The main reductions, size-bounded hardness packages, exact finite boolean deciders, counted finite-search procedures, and explicit-state step-counting wrappers for the regime-typed query predicates are mechanized in Lean 4. For the stochastic existential variants, the artifact now also mechanizes the witness-over-verifier schema, the existential-majority reduction core, and an internal completeness-style packaging for the stochastic anchor family (*L: [OU1-2](#), [EH1-2](#), [EH4-5](#).*) Standard PP/PSPACE and oracle-class membership arguments still remain part of the paper text rather than end-to-end Turing-machine witness proofs in the current repository.

1.2 Quantifier Structure

The complexity gap comes from the structure of the question. Insufficiency has short witnesses: two states that agree on the proposed coordinates but induce different optimal-action sets. Sufficiency, by contrast, asks for the absence of every such witness. So the object-level task is forward evaluation on a fully specified state, while the meta-level task is universal verification over counterexample pairs. This difference drives the coNP classification and explains why relevance identification is not just another forward-evaluation problem.

The static regime also contains the paper’s main structural simplification. Although minimum sufficiency can be written with an outer existential quantifier over coordinate sets, the existential layer collapses once sufficiency is characterized as containment of all relevant coordinates. The stochastic and sequential regimes do not simplify in the same way: conditioning introduces majority-style comparisons over fibers, while temporal evolution introduces policy-level dependence on hidden information.

1.3 Formalization Snapshot

Metric	Value
Lines of Lean 4 backing cited content	19358
Theorems/lemmas in cited-content files	900
Proof files backing cited content	57
sorry count in cited-content files	0

The artifact builds with `lake build`. It certifies the reduction-correctness lemmas, size-bounded hardness packages, exact finite deciders, counted-search witnesses, tractability statements, explicit-state step-counting wrappers, and the scoped witness-over-verifier and existential-hardness packages cited for the stochastic NP^{PP} boundary (L: [OU1-2](#), [EH2-5](#).) Standard PP/PSPACE and oracle-class membership directions remain paper proofs rather than full Turing-machine formalizations.

1.4 Paper Structure

Section 2 introduces the abstract setup and the encoding contract. Sections 3, 4, 5, and 6 develop the paper’s core theorem package: the Static Collapse Theorem, the regime-sensitive complexity matrix, and the resulting hierarchy from static to stochastic to sequential certification. Section 7 gives the encoding-sensitive dichotomy and the sharp phase transition for exact certification, and Section 8 records tractable subcases. Section 9 then collects the main implications for exact simplification, the trilemma of exact certification, witness checking, approximation, and externalized relevance. Section 10 situates the work within formalized complexity theory and decision-theoretic relevance.

1.5 Artifact Availability

The standalone Lean proof artifact and submission snapshot are archived at:

<https://doi.org/10.5281/zenodo.18140966>

The proofs build with the Lean toolchain specified in `lean-toolchain`.

2 Formal Setup

This section fixes the abstract vocabulary used throughout the paper. We work with finite decision problems whose state spaces factor into coordinates, and we isolate the minimal terminology needed to state the later complexity results. The same setup will support the paper’s three core regime classifications, the encoding-sensitive boundary, and the later consequence sections.

2.1 Decision Problems with Coordinate Structure

Definition 2.1 (Decision Problem). A *decision problem with coordinate structure* is a tuple $\mathcal{D} = (A, X_1, \dots, X_n, U)$ where:

- A is a finite set of *actions* (alternatives)
- $X_1, \dots, X_n$ are finite *coordinate spaces*
- $S = X_1 \times \dots \times X_n$ is the *state space*
- $U : A \times S \rightarrow \mathbb{Q}$ is the *utility function*

Definition 2.2 (Projection). For state $s = (s_1, \dots, s_n) \in S$ and coordinate set $I \subseteq \{1, \dots, n\}$:

$$s_I := (s_i)_{i \in I}$$

is the *projection* of s onto coordinates in I .

Definition 2.3 (Optimizer Map). For state $s \in S$, the *optimal action set* is:

$$\text{Opt}(s) := \arg \max_{a \in A} U(a, s) = \{a \in A : U(a, s) = \max_{a' \in A} U(a', s)\}$$

2.2 Sufficiency and Relevance

Definition 2.4 (Sufficient Coordinate Set). A coordinate set $I \subseteq \{1, \dots, n\}$ is *sufficient* for decision problem $\mathcal{D}$ if:

$$\forall s, s' \in S : s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s')$$

Equivalently, the optimal action depends only on coordinates in I .

Proposition 2.5 (Empty-Set Sufficiency Equals Constant Decision Boundary). *The empty set $\emptyset$ is sufficient if and only if $\text{Opt}(s)$ is constant over the entire state space.*

(L: [DP6](#).)

Proof. If $\emptyset$ is sufficient, then every two states agree on their empty projection, so sufficiency forces $\text{Opt}(s) = \text{Opt}(s')$ for all $s, s' \in S$. The converse is immediate. ■

Proposition 2.6 (Insufficiency Equals Counterexample Witness). *Coordinate set I is not sufficient if and only if there exist states $s, s' \in S$ such that $s_I = s'_I$ but $\text{Opt}(s) \neq \text{Opt}(s')$.*

(L: [DP7](#).)

Proof. This is the direct negation of Definition 2.4. ■

Definition 2.7 (Minimal Sufficient Set). A sufficient set I is *minimal* if no proper subset $I' \subsetneq I$ is sufficient.

Definition 2.8 (Relevant Coordinate). Coordinate i is *relevant* if there exist states that differ only at coordinate i and induce different optimal-action sets:

$$i \text{ is relevant} \iff \exists s, s' \in S : \left(\forall j \neq i, s_j = s'_j \right) \wedge \text{Opt}(s) \neq \text{Opt}(s').$$

Proposition 2.9 (Minimal-Set/Relevance Equivalence). *For any minimal sufficient set I and any coordinate i ,*

$$i \in I \iff i \text{ is relevant}.$$

Hence every minimal sufficient set is exactly the relevant-coordinate set.

(L: DP1-2.)

Proof. If $i \in I$ were not relevant, then changing coordinate i while keeping all others fixed would never alter the optimal-action set, so removing i would preserve sufficiency, contradicting minimality. Conversely, every sufficient set must contain each relevant coordinate, because omitting a relevant coordinate leaves a witness pair with identical I -projection but different optimal-action sets. ■

Definition 2.10 (Exact Relevance Identifiability). For a decision problem $\mathcal{D}$ and candidate set I , we say that I is *exactly relevance-identifying* if

$$\forall i \in \{1, \dots, n\} : i \in I \iff i \text{ is relevant for } \mathcal{D}.$$

Definition 2.11 (Structural Rank). The *structural rank* of a decision problem is the number of relevant coordinates:

$$\text{srnk}(\mathcal{D}) := |\{i \in \{1, \dots, n\} : i \text{ is relevant}\}|.$$

2.3 Optimizer Quotient

Definition 2.12 (Decision Equivalence). States $s, s' \in S$ are *decision-equivalent* if they induce the same optimal-action set:

$$s \sim_{\text{Opt}} s' \iff \text{Opt}(s) = \text{Opt}(s').$$

Definition 2.13 (Decision Quotient). The *decision quotient* (equivalently, optimizer quotient) of $\mathcal{D}$ is the quotient object

$$Q_{\mathcal{D}} := S / \sim_{\text{Opt}}.$$

Its equivalence classes are exactly the maximal subsets of states on which the optimal-action set is constant.

Proposition 2.14 (Optimizer Quotient as Coimage/Image Factorization). *The decision quotient $Q_{\mathcal{D}}$ is canonically in bijection with $\text{range}(\text{Opt})$. Equivalently, in **Set**, it is the coimage/image factorization of the optimizer map.*

(L: DQ6-7.)

Proof. Two states are identified by $\sim_{\text{Opt}}$ exactly when they have the same optimizer value. Hence quotient classes are in one-to-one correspondence with attained values of Opt . ■

Theorem 2.15 (Universal Property of the Optimizer Quotient). *Let $Q_{\mathcal{D}} = S / \sim_{\text{Opt}}$ be the decision quotient with quotient map $\pi : S \rightarrow Q_{\mathcal{D}}$. For any surjective abstraction $\phi : S \rightarrow T$ such that*

$$\phi(s) = \phi(s') \implies \text{Opt}(s) = \text{Opt}(s'),$$

there exists a unique map $\psi : T \rightarrow Q_{\mathcal{D}}$ such that

$$\pi = \psi \circ \phi.$$

(L: QT1, QT7.)

Proof. Because ϕ preserves optimizer values on its fibers, any two states identified by ϕ are decision-equivalent. So each fiber of ϕ is contained in a single $\sim_{\text{Opt}}$ -class.

For each $t \in T$, choose any $s \in S$ with $\phi(s) = t$ and define $\psi(t) := \pi(s)$. This is well defined because any two such choices lie in the same $\sim_{\text{Opt}}$ -class. By construction, $(\psi \circ \phi)(s) = \pi(s)$ for every $s \in S$, so $\pi = \psi \circ \phi$. Uniqueness follows from surjectivity of ϕ : if ψ' also satisfies $\pi = \psi' \circ \phi$, then every $t \in T$ has the form $t = \phi(s)$ and therefore

$$\psi(t) = \psi(\phi(s)) = \pi(s) = \psi'(\phi(s)) = \psi'(t).$$

Hence $\psi = \psi'$. ■

Remark 2.16 (Canonical Status). The optimizer quotient is therefore the coarsest abstraction of the state space that preserves optimal-action distinctions. Any surjective abstraction that is strictly coarser must erase some decision-relevant distinction.

(L: QT7, AB2.)

Proposition 2.17 (Sufficiency Characterization). *Coordinate set I is sufficient if and only if the optimizer map factors through the projection $\pi_I : S \rightarrow S_I$. Equivalently, there exists a function $\overline{\text{Opt}}_I : S_I \rightarrow \mathcal{P}(A)$ such that*

$$\text{Opt} = \overline{\text{Opt}}_I \circ \pi_I.$$

(L: DQ42.)

Proof. If I is sufficient, define $\overline{\text{Opt}}_I(x)$ to be $\text{Opt}(s)$ for any state s with $s_I = x$; this is well defined exactly because sufficiency makes Opt constant on each projection fiber. Conversely, any such factorization forces states with equal I -projection to have equal optimal-action sets. ■

2.4 Computational Model and Input Encoding

The same decision question yields different computational problems under different input models. We therefore fix the encoding regime before stating any complexity classification.

Succinct encoding (primary for hardness). An instance is encoded by:

- a finite action set A given explicitly,
- coordinate domains $X_1, \dots, X_n$ given by their sizes in binary,
- a Boolean or arithmetic circuit C_U that on input (a, s) outputs $U(a, s)$.

The input length is

$$L = |A| + \sum_i \log |X_i| + |C_U|.$$

Unless stated otherwise, all class-theoretic hardness results in this paper are measured in L .

Explicit-state encoding. The utility function is given as a full table over $A \times S$. The input length is

$$L_{\text{exp}} = \Theta(|A||S|)$$

up to the bitlength of the utility entries. Polynomial-time claims stated in terms of $|S|$ use this explicit-state regime.

Query-access regime. The solver is given oracle access to $\text{Opt}(s)$ or to the utility values needed to reconstruct it. Complexity is then measured by query count, optionally paired with per-query evaluation cost. This regime separates access obstruction from full-table availability and from succinct circuit input length.

Remark 2.18 (Decision Questions vs Decision Problems). The predicate “is coordinate set I sufficient for $\mathcal{D}$?” is an encoding-independent mathematical question. A *decision problem* arises only after fixing how $\mathcal{D}$ and I are represented. Complexity classes are properties of these encoded problems, not of the abstract predicate in isolation.

3 Static Regime Complexity

This section studies the static regime, where the input is a finite coordinate-structured decision problem under the succinct encoding of Section 2.4. This is the baseline certification setting: no stochastic averaging, no temporal dynamics, only the exact question of whether a candidate coordinate set preserves the optimizer map. The main structural fact appears already here: MINIMUM-SUFFICIENT-SET looks like an existential-universal search problem, but it collapses once sufficiency is characterized by relevance containment. The only genuinely higher-level static query is the anchor variant, where an existentially chosen fiber must satisfy a universal constancy condition.

3.1 Problem Definitions

Problem 3.1 (SUFFICIENCY-CHECK). **Input:** Decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$ and coordinate set $I \subseteq \{1, \dots, n\}$.

Question: Is I sufficient for $\mathcal{D}$?

Problem 3.2 (MINIMUM-SUFFICIENT-SET). **Input:** Decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$ and integer k .

Question: Does there exist a sufficient set I with $|I| \leq k$?

Problem 3.3 (ANCHOR-SUFFICIENCY). **Input:** Decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$ and fixed coordinate set I .

Question: Does there exist an assignment α to I such that $\text{Opt}(s)$ is constant on the fiber $\{s : s_I = \alpha\}$?

3.2 The Static Collapse

At first glance, MINIMUM-SUFFICIENT-SET seems to have the form

$$\exists I \forall s, s' \in S : s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s').$$

If that syntax reflected the true structure of the problem, one might expect a Σ_2^P classification. But Proposition 2.9 removes the outer search layer: a set is sufficient exactly when it contains every relevant coordinate, so the minimum sufficient set is simply the relevant-coordinate set. We refer to this phenomenon as the *static collapse*. Static exact minimization is therefore a relevance-counting problem, not a genuinely alternating search problem.

That collapse is specific to minimum sufficiency. ANCHOR-SUFFICIENCY still asks the algorithm to choose an assignment α and then certify a universal condition on the induced fiber, so its existential layer cannot be absorbed into a structural characterization of the instance.

3.3 coNP-Completeness Results

The first two results show that exact certification is already harder than pointwise evaluation. A counterexample to sufficiency is short, but certifying that no such counterexample exists is a universal statement over state pairs. The collapse discussed above explains why the same coNP behavior governs both direct sufficiency checking and exact minimization.

Theorem 3.4 (SUFFICIENCY-CHECK is coNP-complete). *SUFFICIENCY-CHECK is coNP-complete.*

(L: [DQ40-41](#).)

Proof. For membership, Proposition 2.6 gives a polynomial witness (s, s') with $s_I = s'_I$ and $\text{Opt}(s) \neq \text{Opt}(s')$ whenever I is not sufficient, so the complement is in NP.

For hardness, reduce TAUTOLOGY to the empty-set instance. Given formula $\varphi(x_1, \dots, x_n)$, build a two-action decision problem where one action tracks φ and the other is a baseline action. Then $I = \emptyset$ is sufficient iff φ is a tautology, because empty-set sufficiency is exactly constancy of Opt (Proposition 2.5). The reduction is polynomial in formula size. ■

Theorem 3.5 (Static Collapse Theorem). *In the static regime, the apparent existential layer in MINIMUM-SUFFICIENT-SET collapses to relevance containment. Consequently, MINIMUM-SUFFICIENT-SET is coNP-complete.*

(L: [DQ35-36](#).)

Proof. Proposition 2.9 identifies the minimum sufficient set with the relevant-coordinate set. The question “ $|I^*| \leq k$?” therefore becomes the question whether at most k coordinates are relevant, which is a coNP check over relevance witnesses. CoNP-hardness follows from the $k = 0$ slice, which is exactly SUFFICIENCY-CHECK with $I = \emptyset$. ■

3.4 Sigma-2-P Completeness

Anchor sufficiency adds one more layer of choice that the previous collapse does not remove: the algorithm may select a fiber, but must then certify that the chosen fiber is uniformly decision-preserving. That existential-then-universal pattern is exactly what raises the complexity class.

Theorem 3.6 (ANCHOR-SUFFICIENCY is Sigma-2-P-complete). *ANCHOR-SUFFICIENCY is Σ_2^P -complete.*

(L: [DQ21-22](#).)

Proof. Membership follows directly from the quantifier pattern:

$$\exists \alpha \forall s \in S : (s_I = \alpha) \Rightarrow \text{Opt}(s) = \text{Opt}(s_\alpha).$$

For hardness, reduce from $\exists\forall$ -SAT: existential variables encode the anchor assignment, and universal variables range over the residual coordinates. Utility values are set so that fiberwise constancy holds exactly when the source formula is true. ■

3.5 Certificate-Size Lower Bound

The next theorem sharpens the static picture. It is not only that exact certification is hard in the complexity-class sense; even a checker restricted to the empty-set core may be forced to inspect exponentially many witness locations in the worst case.

Theorem 3.7 (Witness-Checking Duality). *For Boolean state spaces $S = \{0,1\}^n$, any sound checker for the empty-set sufficiency core must inspect at least 2^{n-1} witness pairs in the worst case.*

(L: [WD1-3](#).)

Proof. Empty-set sufficiency asks whether Opt is constant on all 2^n states. A sound refutation-complete checker must be able to separate constant maps from maps that differ on some hidden antipodal partition. An adversary can place the first disagreement in any of 2^{n-1} independent pair slots; if fewer than 2^{n-1} slots are inspected, two instances remain indistinguishable to the checker but have opposite truth values. Therefore at least 2^{n-1} pair checks are necessary. ■

Mechanization note. The TAUTOLOGY reduction stack, the coNP and Σ_2^P classifications, and the witness-budget lower-bound core are indexed by their inline Lean handles.

Explicit-state search upper bounds. Under the explicit-state step-counting model, static sufficiency and static anchor sufficiency are also wrapped as abstract P predicates on inputs carrying a certified state budget. The artifact proves these upper bounds via counted searches with quadratic and linear state-space dependence respectively, and it also packages the static sufficiency search as an explicit correctness-plus-step-bound witness and as part of the unified finite-search summary theorem. (L: [DC70-71](#), [DC76](#), [DC80-82](#).)

This completes the baseline regime. The next section keeps the same exact certification question but replaces pairwise counterexample exclusion by conditional expected-utility comparison, which is why the complexity lifts from the static coNP/ Σ_2^P picture to PP.

4 Stochastic Regime Complexity

We now add a distribution over states. This does not weaken the certification problem; it changes its form. Sufficiency is no longer a pointwise optimizer-preservation condition but a statement about conditional optimal decisions under averaging, and that shift is enough to move the problem to PP. The underlying reason is structural: once utilities are compared through conditional expectations, certification becomes a weighted counting problem rather than a pure counterexample-exclusion problem. Static hardness comes from ruling out bad pairs; stochastic hardness comes from deciding which side of a majority-style threshold a conditional expectation lies on.

4.1 Stochastic Decision Problems

Definition 4.1 (Stochastic Decision Problem). A stochastic decision problem is a tuple

$$\mathcal{D}_S = (A, X_1, \dots, X_n, U, P),$$

where $P \in \Delta(S)$ is a distribution on $S = X_1 \times \dots \times X_n$.

Definition 4.2 (Distribution-Conditional Sufficiency). A coordinate set I is *stochastically sufficient* if conditioning on s_I never changes the induced optimal-action set:

$$\arg \max_{a \in A} \mathbb{E}[U(a, s) \mid s_I] = \arg \max_{a \in A} \mathbb{E}[U(a, s)]$$

for all admissible I -fibers.

Problem 4.3 (STOCHASTIC-SUFFICIENCY-CHECK). **Input:** $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ and $I \subseteq \{1, \dots, n\}$.

Question: Is I stochastically sufficient?

Definition 4.4 (Stochastic Anchor Sufficiency). A coordinate set I is *stochastically anchor-sufficient* if there exists an anchor fiber such that the conditional optimal-action set on that fiber is a singleton and every state agreeing with the anchor on I induces that same singleton conditional optimum.

Problem 4.5 (Stochastic Anchor Check). **Input:** $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ and $I \subseteq \{1, \dots, n\}$.

Question: Is I stochastically anchor-sufficient?

Problem 4.6 (Stochastic Minimum Query). **Input:** $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ and $k \in \mathbb{N}$.

Question: Does there exist a stochastically sufficient coordinate set I with $|I| \leq k$?

4.2 PP-Completeness

The stochastic regime preserves the same underlying question, which coordinates can be ignored without changing the decision, but asks it after conditioning and expectation have been introduced. The resulting majority-style comparisons are exactly where PP enters. For the empty-information case, one is already comparing aggregate mass of satisfying versus nonsatisfying states. More generally, every fiberwise exact-certification check asks whether one conditional expected utility dominates another across all admissible fibers, which is the natural counting analogue of the static universal check.

Theorem 4.7 (Stochastic Sufficiency is PP-complete). *STOCHASTIC-SUFFICIENCY-CHECK is PP-complete.*

(L: [DC45](#), [DC62](#), [DC72](#).)

Proof. For membership, deciding whether one conditional expected utility dominates another is a majority-style weighted counting comparison, hence in PP.

For hardness, reduce MAJSAT. Given Boolean formula φ , take $S = \{0, 1\}^n$, uniform P , actions $\{\text{accept}, \text{reject}\}$, and utilities

$$U(\text{accept}, s) = \mathbf{1}[\varphi(s)], \quad U(\text{reject}, s) = 1/2,$$

with $I = \emptyset$. Then empty-set stochastic sufficiency is equivalent to deciding whether $\mathbb{E}[\varphi] \geq 1/2$, i.e., MAJSAT. ■

Proposition 4.8 (Stochastic Sufficiency Refines to Stochastic Anchor Sufficiency). *If I is stochastically sufficient, then I is stochastically anchor-sufficient.*

(L: [DC19](#).)

Proof. If every admissible I -fiber has a unique conditional optimum, then any chosen anchor fiber has a unique conditional optimum, and every state agreeing with that anchor on I induces the same singleton conditional optimum. ■

Theorem 4.9 (Stochastic Anchor Hardness). *STOCHASTIC-ANCHOR-SUFFICIENCY-CHECK is PP-hard.*

(L: [DC40-42](#).)

Proof. Reduce MAJSAT. Given Boolean formula φ on $n \geq 1$ variables, take $S = \{0, 1\}^n$, uniform P , $I = \emptyset$, and three actions $\{\text{accept}, \text{hold}_L, \text{hold}_R\}$. Set

$$U(\text{accept}, s) = \mathbf{1}[\varphi(s)], \quad U(\text{hold}_L, s) = U(\text{hold}_R, s) = \frac{1}{2} - 2^{-(n+1)}.$$

If at least half of all assignments satisfy φ , then accept is the unique optimal action on the empty-information fiber, so the instance is anchor-sufficient. If fewer than half satisfy φ , then hold_L and hold_R tie above accept, so no singleton anchor optimum exists. Thus MAJSAT many-one reduces to the stochastic anchor query. ■

Theorem 4.10 (Stochastic Minimum Hardness). *The stochastic minimum-sufficient-set query is PP-hard.*

(L: [DC45](#), [DC47](#).)

Proof. Reduce MAJSAT to the $k = 0$ slice. In the same three-action gadget used above, there exists a stochastically sufficient set of size at most 0 iff the empty set itself is stochastically sufficient. This empty-set equivalence for the gadget is certified in the artifact, so MAJSAT many-one reduces to STOCHASTIC-MINIMUM-SUFFICIENT-SET. ■

Proposition 4.11 (Explicit Finite Search for Stochastic Queries). *For finite instances, the artifact contains counted exhaustive-search procedures for stochastic sufficiency, stochastic anchor sufficiency, and stochastic minimum sufficiency. Their certified step bounds are respectively $O(|S|)$, $O(|S||A|)$, and $O(2^n)$.*

(L: [DC62-65](#), [DC72-73](#), [DC76](#).)

Proof. The sufficiency search scans for a violating state whose fiber-optimal set is non-singleton; the anchor search scans over candidate anchor-state/action pairs; and the minimum query scans the subset lattice. Each procedure admits a counted boolean search formulation with both a correctness theorem and an explicit step bound. ■

Theorem 4.12 (Stochastic Existential Queries Sit on the NP^{PP} Boundary). *Under the present encoding, STOCHASTIC-ANCHOR-SUFFICIENCY-CHECK and STOCHASTIC-MINIMUM-SUFFICIENT-SET are both in NP^{PP} .*

(L: [OU1-2](#), [OU6-7](#), [PA3](#), [EH1-5](#).)

Proof. For anchor sufficiency, the mechanized collapse

$$\text{STOCHASTIC-ANCHOR-SUFFICIENCY-CHECK}(P, I) \iff \exists s_0 \exists a (\text{fiberOpt}(P, I, s_0) = \{a\})$$

is already available in the artifact [PA3](#). A nondeterministic polynomial-time machine may therefore guess (s_0, a) and use a PP oracle to verify that a is the unique conditional optimum on the anchor

fiber. Concretely, uniqueness can be checked by comparing the conditional expected utility of a against every competing action on that fiber, which is exactly the same majority-style conditional comparison used in the PP membership proof for stochastic sufficiency.

For the minimum query, the machine guesses a coordinate set I with $|I| \leq k$ and then asks whether I is stochastically sufficient. By Theorem 4.7, that verification predicate lies in PP. Hence the minimum query is also in NP^{PP} . ■

Theorem 4.12 completes the paper-level stochastic landscape: stochastic sufficiency is PP-complete, while stochastic anchor and minimum occupy the existential counting boundary between PP-hardness and NP^{PP} membership. The artifact does not package a full oracle-Turing-machine formalization of NP^{PP} . What it does mechanize is the finite core behind that classification: the anchor-collapse equivalence, the generic existential-witness schema, bounded-witness recoveries for the explicit-state stochastic anchor and minimum searches, and an existential-majority reduction package giving an internal completeness-style surrogate for the stochastic anchor family (L: EH2-5.)

Relevance boundary. The stochastic query is fiber-indexed: fixing a candidate information set I induces a derived decision problem whose optimizer is the conditional fiber optimizer. In the current formalization this induced problem is always I -sufficient (L: DC57), but its relevance notion is therefore also indexed by I . For that reason, the paper does not assert a single global relevance-set characterization for stochastic minimal sufficiency analogous to the static and sequential regimes.

4.3 Tractable Subcases

Proposition 4.13 (Tractable Stochastic Cases). *The stochastic sufficiency problem is polynomial-time solvable under each of the following restrictions:*

1. *product distributions* $P = \bigotimes_i P_i$,
2. *bounded support* $|\text{supp}(P)| \leq k$,
3. *structured families where conditional expectations are computable in polynomial time (e.g. log-concave models with oracle access).*

(L: DQ13, DQ15.)

Proof. In each case, conditional expected utilities can either be computed exactly or reduced to a polynomial number of marginal comparisons. ■

4.4 Bridge from Static

These final propositions show that moving from static to stochastic reasoning is not merely a reformulation. Distributional conditioning can genuinely destroy or preserve sufficiency depending on the structure of the input model.

Proposition 4.14 (Static Sufficiency Does Not Imply Stochastic Sufficiency). *There exist instances where I is sufficient in the static sense but not stochastically sufficient once a distribution P is fixed.*

(L: DQ58.)

Proof. Choose utilities where optimizer ties are harmless pointwise but are broken in expectation after conditioning. Then static fibers preserve Opt, while conditional expectations change the optimal action. ■

Proposition 4.15 (Transfer Under Product Distributions). *If P factorizes as a product distribution and I is statically sufficient, then I is stochastically sufficient.*

(L: [DQ15-16](#).)

Proof. Under product factorization, conditioning on I does not introduce cross-coordinate dependence outside the sufficient projection, so optimizer preservation transfers from pointwise to expectation form. ■

The stochastic regime therefore gives the middle layer of the paper’s hierarchy: conditioning already raises exact certification beyond the static setting by turning exact certification into a counting comparison problem, but the full temporal escalation is deferred until sequential dynamics are added in the next section.

5 Sequential Regime Complexity

We now move to sequential settings with transitions and observations. Here the same certification question is asked at the policy level: can one suppress some coordinates of the evolving information state without changing the optimal policy? This is where the complexity reaches PSPACE. The structural reason is that hidden information can matter later even when it appears irrelevant now. Certification must therefore compare policies across temporally unfolding contingencies rather than compare one-shot optimizers or conditional expectations. In succinct models, that policy-preservation task naturally supports the same kind of alternating contingency that drives PSPACE hardness in planning and quantified-logic reductions.

5.1 Sequential Decision Problems

Definition 5.1 (Sequential Decision Problem). A sequential decision problem is a tuple

$$\mathcal{D}_{\text{seq}} = (A, X_1, \dots, X_n, U, T, O),$$

with state space $S = X_1 \times \dots \times X_n$, transition kernel $T : A \times S \rightarrow \Delta(S)$, and observation model $O : S \rightarrow \Delta(\Omega)$.

Definition 5.2 (Sequential Sufficiency). Coordinate set I is *sequentially sufficient* if the optimal policy using full observation history equals the optimal policy obtained when histories are restricted to I -coordinates.

Problem 5.3 (SEQUENTIAL-SUFFICIENCY-CHECK). **Input:** $\mathcal{D}_{\text{seq}}$ and $I \subseteq \{1, \dots, n\}$.

Question: Is I sequentially sufficient?

Definition 5.4 (Sequential Anchor Sufficiency). A coordinate set I is *sequentially anchor-sufficient* if there exists an anchor state whose I -agreement class preserves the optimal action set throughout that class.

Problem 5.5 (SEQUENTIAL-ANCHOR-SUFFICIENCY-CHECK). **Input:** $\mathcal{D}_{\text{seq}}$ and $I \subseteq \{1, \dots, n\}$.

Question: Is I sequentially anchor-sufficient?

Problem 5.6 (Sequential Minimum Query). **Input:** $\mathcal{D}_{\text{seq}}$ and $k \in \mathbb{N}$.

Question: Does there exist a sequentially sufficient coordinate set I with $|I| \leq k$?

Proposition 5.7 (Sequential Minimality Equals Relevance). *For any minimal sequentially sufficient set I , a coordinate lies in I if and only if it is relevant for the underlying decision boundary.*

(L: [DC49-50](#).)

Proof. In the current formalization, sequential sufficiency is defined as sufficiency of the underlying one-step optimizer map on the state space. The static minimal-set/relevance equivalence therefore transports directly to the sequential setting. ■

5.2 PSPACE-Completeness

The sequential regime is the strongest of the three because hidden information can matter not only for the current optimizer but for the future evolution of the decision problem. The same relevance question therefore becomes a policy-comparison problem over dynamic information states. Static certification asks whether bad state pairs exist; stochastic certification asks whether conditional averages cross a threshold; sequential certification asks whether omitted coordinates can change the value of future contingent choices. That added temporal contingency is exactly what the TQBF reduction exploits.

Theorem 5.8 (Sequential Sufficiency is PSPACE-complete). *SEQUENTIAL-SUFFICIENCY-CHECK is PSPACE-complete.*

(L: [DC46](#), [DC66](#), [DC74](#).)

Proof. For membership, finite-horizon policy comparison for POMDP-style models is decidable in polynomial space by dynamic programming over belief and state summaries.

For hardness, reduce TQBF. Encode alternating quantifiers as alternating controlled and adversarial transitions, and let the terminal reward evaluate the quantified formula. Then full-history sufficiency holds exactly when the quantified instance is true. ■

Proposition 5.9 (Sequential Sufficiency Refines to Sequential Anchor Sufficiency). *If I is sequentially sufficient, then I is sequentially anchor-sufficient.*

(L: [DC23](#).)

Proof. If optimal-action sets are preserved for every pair of states agreeing on I , then fixing any anchor state immediately yields preservation across its entire I -agreement class. ■

Theorem 5.10 (Sequential Anchor Query is PSPACE-complete). *The sequential anchor query is PSPACE-complete.*

(L: [DC28-29](#), [DC44](#).)

Proof. Membership is in PSPACE. By definition, the query asks whether there exists an anchor state s_0 such that every state agreeing with s_0 on I preserves the same optimal-action set. A nondeterministic polynomial-space procedure can guess s_0 and then scan candidate states one at a time, verifying the agreement condition and comparing the corresponding optimal-action sets. Since $\text{NPSpace} = \text{PSPACE}$, the query lies in PSPACE.

Reuse the TQBF reduction for sequential sufficiency with $I = \emptyset$. At empty information, sequential anchor sufficiency is equivalent to global preservation of the optimal-action set, so the same construction yields

$$\text{TQBF}(q) \iff \text{SEQUENTIAL-ANCHOR-SUFFICIENCY-CHECK}(\text{reduceTQBF}(q), \emptyset).$$

Hence TQBF many-one reduces to the sequential anchor query. ■

Theorem 5.11 (Sequential Minimum Query is PSPACE-complete). *The sequential minimum-sufficient-set query is PSPACE-complete.*

(L: [DC46](#), [DC48](#).)

Proof. Membership is in PSPACE. The query asks whether there exists a coordinate set I of size at most k such that I is sequentially sufficient. A nondeterministic polynomial-space procedure can guess I and then run the same polynomial-space verifier used for sequential sufficiency. Again, $\text{NPSPACE} = \text{PSPACE}$.

Reduce TQBF to the $k = 0$ slice. There exists a sequentially sufficient set of size at most 0 iff the empty set itself is sequentially sufficient. The same construction yields PSPACE-hardness for SEQUENTIAL-MINIMUM-SUFFICIENT-SET. ■

Proposition 5.12 (Explicit Finite Search for Sequential Queries). *For finite instances, the development contains counted exhaustive-search procedures for sequential sufficiency, sequential anchor sufficiency, and sequential minimum sufficiency. Their certified step bounds are respectively $O(|S|^2)$, $O(|S|)$, and $O(2^n)$.*

(L: [DC66-69](#), [DC74-76](#).)

Proof. The sufficiency search scans for an agreeing pair of states with different optimal sets; the anchor search scans over candidate anchor states; and the minimum query scans the subset lattice. Each procedure admits a counted boolean search formulation with both a correctness theorem and an explicit step bound. ■

5.3 Tractable Subcases

Proposition 5.13 (Tractable Sequential Cases). *The sequential sufficiency problem is polynomial-time solvable under each of the following restrictions:*

1. *full observability (MDP case),*
2. *bounded horizon,*
3. *tree-structured transition systems,*
4. *deterministic transitions.*

(L: [DQ12](#), [DQ14](#), [DQ45](#).)

Proof. Each restriction reduces policy search to dynamic programming over a polynomially bounded representation. ■

5.4 Bridge from Stochastic

The bridge from the stochastic regime is strict. A one-shot distribution can hide no temporal contingency; once dynamics are introduced, latent information can become decision-relevant later even when it appears irrelevant at time zero.

Proposition 5.14 (Stochastic Sufficiency Does Not Imply Sequential Sufficiency). *There exist instances where I is sufficient in the stochastic one-shot sense but fails to be sufficient once temporal dynamics are introduced.*

(L: [DQ62.](#))

Proof. Construct a process in which the same one-step expected optimizer is preserved under I , but hidden-state memory affects future information states and therefore optimal policies. The temporal dependence blocks one-shot sufficiency transfer. ■

With the static, stochastic, and sequential classifications in place, the next section consolidates them into a single regime hierarchy and makes the escalation explicit: counterexample exclusion, then counting comparison, then policy preservation over time.

6 Regime Hierarchy

The static, stochastic, and sequential formulations are the paper’s central organizing object. The same exact certification question is posed in three increasingly expressive models, and the resulting complexity classes separate accordingly. Read as a complexity matrix, the addition of probability lifts exact certification from coNP to PP, and the addition of time lifts it again to PSPACE.

6.1 Complexity Matrix

Regime	Sufficiency	Minimum	Anchor
Static	coNP-complete	coNP-complete	Σ_2^P -complete
Stochastic	PP-complete	PP-hard	PP-hard
Sequential	PSPACE-complete	PSPACE-complete	PSPACE-complete

This table is the paper’s compact summary. Adding probability lifts exact certification from coNP to PP, and adding time lifts it again to PSPACE. The stochastic row is complete at the paper level even though its existential cells are expressed as a boundary classification rather than a collapsed single class: sufficiency is PP-complete, while anchor and minimum are pinned between PP-hardness and NP^{PP} membership.

The stochastic anchor and minimum variants are not presently collapsed to PP-completeness: Theorem 4.12 places them in NP^{PP} , while Theorems 4.9 and 4.10 give PP-hardness. The artifact now mechanizes the witness-over-verifier structure and explicit-state bounded-witness recoveries behind that upper-bound story, and it also packages an existential-majority reduction and internal completeness-style surrogate for the stochastic anchor family (L: [EH2-5.](#)) It still does not package a full oracle-class membership proof under the succinct encoding. By contrast, the sequential anchor and minimum variants admit paper-level PSPACE membership via guess-and-verify over anchors or coordinate subsets together with the sequential sufficiency verifier.

Theorem 6.1 (Unified Regime Hierarchy for Exact Relevance Certification). *Under the encoding model of Section 2.4, exact relevance certification admits a strict regime-sensitive hierarchy. In the static regime, sufficiency and minimum sufficiency are coNP-complete and anchor sufficiency is Σ_2^P -complete. In the stochastic regime, sufficiency is PP-complete, while the minimum and anchor queries lie on the existential counting boundary: they are PP-hard and admit paper-level NP^{PP} upper bounds. In the sequential regime, sufficiency, minimum, and anchor queries are PSPACE-complete. Under standard complexity assumptions, this yields strict inclusions*

$$\text{static} \subset \text{stochastic} \subset \text{sequential}.$$

(L: [DQ1](#), [DQ17-19](#), [OU1-2](#), [EH4-5.](#))

Proof. The matrix entries are exactly Theorems 3.4, 3.5, 3.6, 4.7, 4.9, 4.10, 5.8, 5.10, and 5.11. The strict inclusions then follow from the standard class separations invoked in Theorems 6.2 and 6.3. ■

6.2 Strict Inclusions

Theorem 6.2 (Static $\subset$ Stochastic). *Under standard assumptions (in particular $P \neq \text{coNP}$), the stochastic regime strictly extends the static regime in computational complexity.*

(L: DQ18.)

Proof. Static sufficiency checking is coNP-complete (Theorem 3.4), while stochastic sufficiency checking is PP-complete (Theorem 4.7). Under the stated assumptions, this yields strict separation in regime difficulty. ■

Theorem 6.3 (Stochastic $\subset$ Sequential). *Under standard assumptions (in particular $P \neq \text{PP}$), the sequential regime strictly extends the stochastic regime in computational complexity.*

(L: DQ19.)

Proof. Stochastic sufficiency is PP-complete (Theorem 4.7), while sequential sufficiency is PSPACE-complete (Theorem 5.8). The class gap yields strict regime inclusion under the stated assumptions. ■

6.3 Integrity-Resource Bounds

The hierarchy has a methodological consequence as well: as one moves upward in the hierarchy, exact certification becomes less compatible with fixed polynomial resource budgets.

Proposition 6.4 (Abstention Frontier Expands with Regime Complexity). *For polynomial-time exact certifiers that abstain whenever they cannot certify correctness within budget, the set of instances requiring abstention expands along*

$$\text{static} \rightarrow \text{stochastic} \rightarrow \text{sequential}.$$

(L: IC32, DQ2.)

Proof. As the decision predicate moves from coNP to PP to PSPACE hardness, exact worst-case certification requires strictly stronger resources. Fixed polynomial resources therefore force abstention on a larger instance family in higher regimes. ■

7 Encoding Dichotomy and ETH Lower Bounds

The regime hierarchy identifies where exact relevance certification sits in the standard complexity landscape. This section extracts the complementary algorithmic message: under the encodings of Section 2.4, there is a genuine dichotomy between explicit-state instances whose true relevant support is small and succinct instances whose relevant support is extensive. The former admit direct algorithms; the latter inherit exponential lower bounds under ETH. The practical takeaway is a sharp phase transition: logarithmic relevant support can still be certified exactly in polynomial time under explicit encodings, while linear relevant support under succinct encodings already forces ETH-conditioned exponential cost.

Model note. Part 1 is an explicit-state upper bound (time polynomial in $|S|$). Part 2 is a succinct-encoding lower bound under ETH (time exponential in n). The encodings are defined in Section 2.4.

Theorem 7.1 (Complexity Dichotomy). *Let $\mathcal{D} = (A, X_1, \dots, X_n, U)$ be a decision problem with $|S| = N$ states. Let k^* be the size of the minimal sufficient set.*

1. **Logarithmic case (explicit-state upper bound):** *If $k^* = O(\log N)$, then SUFFICIENCY-CHECK is solvable in polynomial time in N under the explicit-state encoding.*
2. **Linear case (succinct lower bound under ETH):** *If $k^* = \Omega(n)$, then SUFFICIENCY-CHECK requires time $\Omega(2^{n/c})$ for some constant $c > 0$ under the succinct encoding (assuming ETH).*

(L: [DQ26](#), [DQ29-30](#).)

Proof. Part 1 (Logarithmic case): If $k^* = O(\log N)$, then the number of distinct projections $|S_{I^*}|$ is at most $2^{k^*} = O(N^c)$ for some constant c . Under the explicit-state encoding, we enumerate all projections and verify sufficiency in polynomial time.

Part 2 (Linear case): We establish this via an explicit reduction chain from the Exponential Time Hypothesis under the succinct encoding. ■

7.1 The ETH Reduction Chain

The lower bound in Part 2 of Theorem 7.1 follows from a chain of reductions originating in the Exponential Time Hypothesis. We make this chain explicit.

Definition 7.2 (Exponential Time Hypothesis (ETH)). There exists a constant $\delta > 0$ such that 3-SAT on n variables cannot be solved in time $O(2^{\delta n})$ [16].

The chain proceeds as follows:

1. **ETH $\Rightarrow$ 3-SAT requires $2^{\Omega(n)}$:** This is the definition of ETH.
2. **3-SAT $\leq_p$ TAUTOLOGY:** Given 3-SAT formula $\varphi(x_1, \dots, x_n)$, define $\psi = \neg\varphi$. Then φ is satisfiable iff ψ is not a tautology. This is a linear-time reduction preserving the number of variables.
3. **TAUTOLOGY requires $2^{\Omega(n)}$ (under ETH):** By the contrapositive of step 2, if TAUTOLOGY is solvable in $o(2^{\delta n})$ time, then 3-SAT is solvable in $o(2^{\delta n})$ time, contradicting ETH.
4. **TAUTOLOGY $\leq_p$ SUFFICIENCY-CHECK:** This is Theorem 3.4. Given formula $\varphi(x_1, \dots, x_n)$, we construct a decision problem where:
 - The empty set $I = \emptyset$ is sufficient iff φ is a tautology
 - When φ is not a tautology, all n coordinates are relevant

The reduction is polynomial-time and preserves the number of coordinates.

5. **SUFFICIENCY-CHECK requires $2^{\Omega(n)}$ (under ETH):** Combining steps 3 and 4: if SUFFICIENCY-CHECK is solvable in $o(2^{\delta n/c})$ time for some constant c , then TAUTOLOGY (and hence 3-SAT) is solvable in subexponential time, contradicting ETH.

Proposition 7.3 (Tight Constant). *The reduction in Theorem 3.4 preserves the number of variables up to an additive constant: an n -variable formula yields an $(n + 1)$ -coordinate decision problem. Therefore, the constant c in the $2^{n/c}$ lower bound is asymptotically 1:*

$$\text{SUFFICIENCY-CHECK requires time } \Omega(2^{\delta(n-1)}) = 2^{\Omega(n)} \text{ under ETH}$$

where δ is the ETH constant for 3-SAT.

(L: [DQ27](#), [DQ30](#).)

Proof. The TAUTOLOGY reduction (Theorem 3.4) constructs:

- State space $S = \{\text{ref}\} \cup \{0, 1\}^n$ with $n + 1$ coordinates (one extra for the reference state)
- Query set $I = \emptyset$

When φ has n variables, the constructed problem has $n + 1$ coordinates. The asymptotic lower bound is $2^{\Omega(n)}$ with the same constant δ from ETH. ■

7.2 Phase Transition

Corollary 7.4 (Phase Transition for Exact Certification). *The tractable-hard boundary for exact certification is sharp at the logarithmic scale (with respect to the encodings in Section 2.4):*

- If $k^* = O(\log N)$, SUFFICIENCY-CHECK is polynomial in N under the explicit-state encoding
- If $k^* = \Omega(n)$, SUFFICIENCY-CHECK is exponential in n under ETH in the succinct encoding

For Boolean coordinate spaces ($N = 2^n$), the transition is between $k^* = O(\log n)$ (explicit-state tractable) and $k^* = \Omega(n)$ (succinct-encoding intractable).

(L: [DQ3](#), [DQ26](#).)

Proof. The logarithmic case (Part 1 of Theorem 7.1) gives polynomial time when $k^* = O(\log N)$. More precisely, when $k^* \leq c \log N$ for constant c , the algorithm runs in time $O(N^c \cdot \text{poly}(n))$.

The linear case (Part 2) gives exponential time when $k^* = \Omega(n)$.

The transition point is where $2^{k^*} = N^{k^*/\log N}$ stops being polynomial in N , i.e., when $k^* = \omega(\log N)$. For Boolean coordinate spaces ($N = 2^n$), this corresponds to the gap between $k^* = O(\log n)$ and $k^* = \Omega(n)$. ■

Remark 7.5 (Sharpness of Dichotomy). Under ETH, the lower bound is asymptotically tight in the succinct encoding. The explicit-state upper bound is tight in the sense that it matches enumeration complexity in N .

This dichotomy explains why exact simplification can be straightforward on some instance families and computationally prohibitive on others. When the true sufficient support is tiny and the state space is explicit, exhaustive certification can be polynomial. When the true support is extensive and the problem is given succinctly, the same certification task inherits a worst-case $2^{\Omega(n)}$ lower bound under ETH in addition to the class-level coNP hardness statement.

Interpretation. The ETH chain composes with the structural rank viewpoint introduced earlier. When relevance is extensive, the same objects that drive the regime hierarchy also force the exponential obstruction in the succinct model. In that sense, the dichotomy theorem complements the hierarchy rather than standing apart from it: the hierarchy locates exact certification in the standard class landscape, while the dichotomy explains when explicit structure can still make exact certification feasible.

Remark 7.6 (Circuit Model Formalization). The ETH lower bound is stated in the succinct circuit encoding of Section 2.4, where the utility function $U : A \times S \rightarrow \mathbb{R}$ is represented by a Boolean circuit computing $\mathbf{1}[U(a, s) > \theta]$ for threshold comparisons. In this model:

- The input size is the circuit size m , not the state space size $|S| = 2^n$
- A 3-SAT formula with n variables and c clauses yields a circuit of size $O(n + c)$
- The reduction preserves instance size up to constant factors: $m_{\text{out}} \leq 3 \cdot m_{\text{in}}$

This linear size preservation is essential for ETH transfer. In the explicit enumeration model (where S is given as a list), the reduction would blow up the instance size exponentially, precluding ETH-based lower bounds. The circuit model is standard in fine-grained complexity and matches practical representations of decision problems.

8 Tractable Special Cases

We distinguish the encodings of Section 2.4. The tractability results below state the model assumption explicitly.

Theorem 8.1 (Tractable Subcases). *SUFFICIENCY-CHECK is polynomial-time solvable in the following cases:*

1. **Bounded actions (explicit-state encoding):** *The input contains the full utility table over $A \times S$ with $|A| \leq k$ for constant k . Complexity: $O(|S|^2 \cdot k^2)$.*
2. **Separable utility (rank 1):** $U(a, s) = f(a) + g(s)$. *Complexity: $O(1)$ (trivially sufficient).*
3. **Low tensor rank:** *The utility admits a rank- R decomposition $U(a, s) = \sum_{r=1}^R w_r \cdot f_r(a) \cdot \prod_i g_{ri}(s_i)$. Complexity: $O(|A| \cdot R \cdot n)$.*
4. **Tree-structured dependencies:** *Coordinates are ordered such that s_i depends only on $(s_1, \dots, s_{i-1})$. Complexity: $O(n \cdot |A| \cdot k_{\max})$ where $k_{\max} = \max_i |X_i|$.*
5. **Bounded treewidth:** *The interaction graph has treewidth $\leq w$ and utility factors over edges. Complexity: $O(n \cdot k^{w+1})$ via standard CSP algorithms.*
6. **Coordinate symmetry:** *Utility is invariant under coordinate permutations. Complexity: $O\left(\binom{d+k-1}{k-1}^2\right)$ orbit-type pairs.*

(L: [DQ44](#), [DQ51](#), [DQ57](#), [DQ63](#), [DQ77](#), [DQ81](#), [DQ85](#).)

The following subsections provide the formal definitions and reduction theorems for each tractable subcase.

8.1 Bounded Actions

When the action set is bounded, brute-force enumeration becomes polynomial.

Definition 8.2 (Bounded Action Problem). A decision problem has *bounded actions* with parameter k if $|A| \leq k$ for some constant k independent of the state space size.

Theorem 8.3 (Bounded Actions Tractability). *If $|A| \leq k$ for constant k , then SUFFICIENCY-CHECK runs in $O(|S|^2 \cdot k^2)$ time.*

(L: [DQ51](#), [DQ72](#).)

Proof. Given the full table of $U(a, s)$:

1. Compute $\text{Opt}(s)$ for all $s \in S$ in $O(|S| \cdot k)$ time.
2. For each pair (s, s') with $s_I = s'_I$, compare $\text{Opt}(s)$ and $\text{Opt}(s')$ in $O(k^2)$ time.
3. Total: $O(|S|^2)$ pairs $\times O(k^2)$ comparisons $= O(|S|^2 \cdot k^2)$.

When k is constant, this is polynomial in $|S|$. ■

8.2 Separable Utility (Rank 1)

Separable utility decouples actions from states, making sufficiency trivial.

Definition 8.4 (Separable Utility). A utility function is *separable* if there exist functions $f : A \rightarrow \mathbb{R}$ and $g : S \rightarrow \mathbb{R}$ such that $U(a, s) = f(a) + g(s)$ for all $(a, s) \in A \times S$.

Theorem 8.5 (Separable Tractability). *If utility is separable, then $I = \emptyset$ is always sufficient.*

(L: [DQ57](#), [DQ74](#).)

Proof. If $U(a, s) = f(a) + g(s)$, then:

$$\text{Opt}(s) = \arg \max_{a \in A} [f(a) + g(s)] = \arg \max_{a \in A} f(a)$$

The optimal action depends only on f , not on s . Hence $\text{Opt}(s) = \text{Opt}(s')$ for all s, s' , making any I (including $\emptyset$) sufficient. ■

8.3 Low Tensor Rank

Utility functions with low tensor rank admit efficient factored computation, generalizing separable utility.

Definition 8.6 (Tensor Rank Decomposition). A utility function $U : A \times ((i : \text{Fin } n) \rightarrow X_i) \rightarrow \mathbb{R}$ has *tensor rank* R if there exist weights $w_1, \dots, w_R \in \mathbb{R}$, action factors $f_r : A \rightarrow \mathbb{R}$, and coordinate factors $g_{ri} : X_i \rightarrow \mathbb{R}$ such that:

$$U(a, s) = \sum_{r=1}^R w_r \cdot f_r(a) \cdot \prod_{i=1}^n g_{ri}(s_i)$$

Remark 8.7. Separable utility (Definition 8.4) is the special case $R = 1$ with $w_1 = 1$, $f_1(a) = f(a)$, and $\prod_i g_{1i}(s_i) = g(s)$.

Theorem 8.8 (Low Tensor Rank Tractability). *If utility has tensor rank R , then SUFFICIENCY-CHECK runs in $O(|A| \cdot R \cdot n)$ time per state.*

(L: [DQ75-77](#).)

Proof. The factored representation enables efficient computation:

1. For each rank component r , the contribution $w_r \cdot f_r(a) \cdot \prod_i g_{ri}(s_i)$ factors over coordinates.
2. Two states s, s' agreeing on I have $g_{ri}(s_i) = g_{ri}(s'_i)$ for $i \in I$.
3. The partial product $\prod_{i \in I} g_{ri}(s_i)$ is identical for agreeing states.
4. This reduces sufficiency checking to comparing factored sums over orbit representatives.

■

8.4 Tree-Structured Dependencies

When coordinate dependencies form a tree, dynamic programming yields polynomial-time sufficiency checking.

Definition 8.9 (Tree-Structured Dependencies). A decision problem has *tree-structured dependencies* if there exists an ordering of coordinates $(1, \dots, n)$ such that the utility decomposes as:

$$U(a, s) = \sum_{i=1}^n u_i(a, s_i, s_{\text{parent}(i)})$$

where $\text{parent}(i) < i$ for all $i > 1$, and the root term depends only on (a, s_1) .

Theorem 8.10 (Tree Structure Tractability). *If dependencies are tree-structured with explicit local tables, SUFFICIENCY-CHECK runs in $O(n \cdot |A| \cdot k_{\max})$ time where $k_{\max} = \max_i |X_i|$.*

(L: [DQ63](#), [DQ78](#).)

Proof. Dynamic programming on the tree order:

1. Process coordinates in order $i = n, n-1, \dots, 1$.
2. For each node i and each value of its parent coordinate, compute the set of actions optimal for some subtree assignment.
3. Combine child summaries with local tables in $O(|A| \cdot k_{\max})$ per node.
4. Total: $O(n \cdot |A| \cdot k_{\max})$.

A coordinate is relevant iff varying its value changes the optimal action set. ■

8.5 Bounded Treewidth

The tree-structured case generalizes to bounded treewidth interaction graphs via CSP reduction.

Definition 8.11 (Interaction Graph). Given a pairwise utility decomposition, the *interaction graph* $G = (V, E)$ has:

- Vertices $V = \{1, \dots, n\}$ (one per coordinate)
- Edges $E = \{(i, j) : i \text{ and } j \text{ interact in the utility}\}$

Definition 8.12 (Pairwise Utility). A utility function is *pairwise* if it decomposes as:

$$U(a, s) = \sum_{i=1}^n u_i(a, s_i) + \sum_{(i,j) \in E} u_{ij}(a, s_i, s_j)$$

with unary terms u_i and binary terms u_{ij} given explicitly.

Theorem 8.13 (Bounded Treewidth Tractability). *If the interaction graph has treewidth $\leq w$ and utility is pairwise with explicit factors, then SUFFICIENCY-CHECK runs in $O(n \cdot k^{w+1})$ time where $k = \max_i |X_i|$.*

(L: [DQ79-81](#).)

Proof. We reduce SUFFICIENCY-CHECK to constraint satisfaction on the interaction graph:

1. **Reduction:** For each action $a \in A$, checking whether $\text{Opt}(s) = \text{Opt}(s')$ for states agreeing on I reduces to a CSP on G with variables $\{s_i : i \notin I\}$.
2. **Standard algorithm:** CSP on graphs of treewidth w is solvable in $O(n \cdot k^{w+1})$ via tree decomposition [4, 11].

The reduction is polynomial; the algorithm is standard. ■

8.6 Coordinate Symmetry

When utility is invariant under coordinate permutations, the effective state space collapses to orbit types.

Definition 8.14 (Dimensional State Space). A *dimensional state space* with alphabet size k and dimension d is $S = \{0, \dots, k-1\}^d$, the set of d -tuples over a k -element alphabet.

Definition 8.15 (Symmetric Utility). A utility function on a dimensional state space is *symmetric* if it is invariant under coordinate permutations: for all permutations $\sigma \in \mathfrak{S}_d$ and states $s \in S$,

$$U(a, s) = U(a, \sigma \cdot s) \quad \text{where } (\sigma \cdot s)_i = s_{\sigma^{-1}(i)}$$

Definition 8.16 (Orbit Type). The *orbit type* of a state $s \in \{0, \dots, k-1\}^d$ is the multiset of its coordinate values:

$$\text{orbitType}(s) = \{\{s_1, s_2, \dots, s_d\}\}$$

Two states have the same orbit type iff they lie in the same orbit under coordinate permutation.

Theorem 8.17 (Orbit Type Characterization). *For dimensional state spaces, two states s, s' have equal orbit types if and only if there exists a coordinate permutation σ such that $\sigma \cdot s = s'$.*

(L: [DQ82](#).)

Proof. **If:** Permuting coordinates preserves the multiset of values. **Only if:** If s and s' have the same multiset of values, we can construct a permutation mapping each occurrence in s to the corresponding occurrence in s' . ■

Theorem 8.18 (Symmetric Sufficiency Reduction). *Under symmetric utility, optimal actions depend only on orbit type:*

$$\text{orbitType}(s) = \text{orbitType}(s') \implies \text{Opt}(s) = \text{Opt}(s')$$

Thus *SUFFICIENCY-CHECK* reduces to checking pairs with different orbit types.

(L: [DQ83-84](#).)

Proof. If $\text{orbitType}(s) = \text{orbitType}(s')$, then by Theorem [8.17](#) there exists σ with $\sigma \cdot s = s'$. By symmetry, $U(a, s) = U(a, s')$ for all a , so $\text{Opt}(s) = \text{Opt}(s')$.

For sufficiency, we need only check pairs (s, s') agreeing on I with *different* orbit types; same-orbit pairs are automatically equal. ■

Theorem 8.19 (Symmetric Complexity Bound). *The number of orbit types is bounded by the stars-and-bars count:*

$$|\text{OrbitTypes}| = \binom{d+k-1}{k-1}$$

Thus *SUFFICIENCY-CHECK* requires at most $\binom{d+k-1}{k-1}^2$ orbit-type pair checks, which is polynomial in d for fixed k .

(L: [DQ56](#), [DQ85](#).)

Proof. An orbit type is a multiset of d values from $\{0, \dots, k-1\}$, equivalently a k -tuple of non-negative integers summing to d . By stars-and-bars, the count is $\binom{d+k-1}{k-1}$.

For fixed k , this is $O(d^{k-1})$, polynomial in d . ■

8.7 Practical Implications

The six tractable cases correspond to common modeling scenarios:

- **Bounded actions:** Small action sets (e.g., binary decisions, few alternatives).
- **Separable utility:** Additive cost models, linear utility functions.
- **Low tensor rank:** Factored preference models, low-rank approximations.
- **Tree structure:** Hierarchical decision processes, causal models with tree structure.
- **Bounded treewidth:** Spatial models, graphical models with limited interaction.
- **Coordinate symmetry:** Exchangeable features, order-invariant utilities.

For problems given in the succinct encoding without these structural restrictions, the hardness results of Section [3](#) apply, justifying heuristic approaches.

9 Implications for Exact Certification

The regime hierarchy matters for more than class labels. The same exact question, which coordinates can be omitted without changing the decision boundary, reappears when one tries to simplify models, build reliable certifiers, check witnesses, approximate exact minimizers, or compress a central interface. This section collects those consequences in one place. The common lesson is that exact relevance is expensive to certify, subject to a trilemma in the hard regime, and easy to displace rather than remove.

9.1 Exact Simplification and Over-Specification

Proposition 9.1 (Configuration Simplification as SUFFICIENCY-CHECK). *Let $P = \{p_1, \dots, p_n\}$ be configuration parameters and let B be a finite set of observable behaviors. Suppose each configuration state determines the subset of behaviors that remain feasible or optimal. Then the question*

“Does parameter subset $I \subseteq P$ preserve all decision-relevant behavior?”

is an instance of SUFFICIENCY-CHECK.

(L: CR1, CT12.)

Proof. Construct a decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$ by taking actions $A = B$, coordinate domains X_i equal to the domains of the parameters p_i , and defining $U(b, s) = 1$ exactly when behavior b is realized or admissible under configuration state s . Then $\text{Opt}(s)$ is the behavior set induced by s , and coordinate subset I is sufficient exactly when agreement on the parameters in I forces agreement on the induced optimal-behavior set. ■

Concrete case study. Consider a simplified service configuration with three parameters: cache mode $p_1 \in \{\text{off}, \text{on}\}$, retry policy $p_2 \in \{1, 3\}$, and replica count $p_3 \in \{1, 2\}$. Let the observable behavior set be $B = \{\text{latency-ok}, \text{write-safe}, \text{cost-ok}\}$. Suppose enabling the cache affects only latency, increasing retries affects only write safety, and replica count affects write safety and cost. Then Theorem 9.1 turns the simplification question into an exact sufficiency question on the induced decision problem: which parameter subsets preserve the optimal-behavior set? In this toy instance, if the target simplification must preserve write safety and cost but is indifferent to latency, the exact behavior-preserving core is $\{p_2, p_3\}$, while p_1 is irrelevant to that target behavior set. The example is small enough to inspect by hand, but it is already an instance of the general Lean-backed reduction from behavior preservation to sufficiency.

(L: CR1, CT12.)

Corollary 9.2 (No General-Purpose Exact Configuration Minimizer). *Assuming $P \neq \text{coNP}$, there is no polynomial-time general-purpose procedure that takes an arbitrary succinctly encoded configuration problem and always returns a smallest behavior-preserving parameter subset.*

(L: DQ36-37.)

Proof. Such a procedure would solve MINIMUM-SUFFICIENT-SET in polynomial time for arbitrary succinctly encoded instances, contradicting Theorem 3.5 under the assumption $P \neq \text{coNP}$. ■

Proposition 9.3 (Model-Conditional Rationality of Over-Specification). *Consider a design process in which:*

1. removing a parameter requires certifying that it is irrelevant,
2. exact irrelevance certification is charged according to the computational model of Section 2.4, and
3. carrying an extra parameter incurs only linear or otherwise low-order maintenance overhead.

Then, for sufficiently expressive succinctly encoded instances, retaining extra parameters can minimize total cost under the stated cost model.

(L: [DQ26](#), [HD14](#).)

Proof. By Theorems 3.4 and 3.5, exact sufficiency certification and exact minimization are coNP-complete in the general succinct regime. By Section 7, this hardness is accompanied by exponential lower bounds under ETH in the linear-support regime. If the cost of carrying extra parameters grows only linearly while exact minimization inherits worst-case exponential cost, then beyond a problem-dependent threshold the latter dominates the former. Under that cost model, retaining extra parameters minimizes total cost. ■

Remark 9.4. This does not rule out useful simplification tools. It rules out a fully general exact minimizer with worst-case polynomial guarantees. The structured regimes isolated in Section 8 remain viable precisely because they restrict the source of hardness.

9.2 Regime-Limited Exact Certification

Proposition 9.5 (Exact Certification Competence Depends on Regime). *Within the model of this paper, exact certification competence is regime-dependent. In the general succinct regime, exact relevance certification and exact minimization inherit the hardness results of Sections 3 and 7. In the structured regimes of Section 8, exact certification becomes available in polynomial time.*

(L: [DQ36](#), [DQ40](#), [DQ44](#).)

Proof. The negative side is given by Theorems 3.4 and 3.5, together with the ETH-conditioned lower bounds summarized in Section 7. The positive side is given by the tractability results of Section 8, which provide explicit polynomial-time certification procedures under structural restrictions. ■

The next three subsections sharpen that basic regime distinction in different directions: exact reliability under polynomial budgets, witness-checking and approximation limits, and the cost of compressing a central interface while leaving exact relevance unresolved.

9.3 Integrity, Competence, and the Trilemma of Exact Certification

Once exact certification is hard, the natural systems question is not only whether the predicate can be computed, but whether a solver can stay exact, non-abstaining, and polynomial-budgeted on the whole hard regime. The theorem below shows that those three goals are incompatible. We refer to this incompatibility as the *trilemma of exact certification*.

Definition 9.6 (Certifying Solver). For exact sufficiency on a declared regime Γ , a certifying solver is a pair (Q, V) where:

- Q maps each instance to either **ABSTAIN** or a candidate verdict with witness,
- V verifies witnesses in polynomial time (in the declared encoding length).

Definition 9.7 (Integrity). A certifying solver is *integrity-preserving* if every non-abstaining output is accepted by V , and every accepted output is correct for the declared exact relation.

Definition 9.8 (Competence). A certifying solver is *competent* on Γ if it is integrity-preserving, non-abstaining on all in-scope instances, and polynomial-budget compliant on all in-scope instances.

Definition 9.9 (Exact Reliability Claim). An *exact reliability claim* on Γ is the conjunction of universal non-abstaining coverage, correctness, and polynomial-budget compliance for exact sufficiency.

Proposition 9.10 (Integrity and Competence Separation). *Integrity and competence are distinct: integrity constrains what can be asserted, while competence adds full coverage under resource bounds.*

(L: [DQ10](#), [IC33](#).)

Proof. The always-abstain solver is integrity-preserving (it makes no uncertified assertion) but fails competence whenever the in-scope set is non-empty. ■

Theorem 9.11 (Integrity-Resource Bound (Conditional)). *Fix a regime Γ whose exact-sufficiency decision problem is coNP-complete under the declared encoding. Under $P \neq \text{coNP}$, no solver is simultaneously integrity-preserving and competent on Γ for exact sufficiency.*

(L: [IC34](#), [DQ31](#).)

Proof. Assume such a solver (Q, V) exists. We build a polynomial-time decider for exact sufficiency on Γ .

Given an input instance $x \in \Gamma$, run $Q(x)$. Competence implies that Q never abstains on in-scope instances and always halts within polynomial budget. Let y be the returned verdict with witness. Now run the polynomial-time verifier $V(x, y)$.

Integrity implies two things: every non-abstaining output produced by Q is accepted by V , and every accepted output is correct for the declared exact-sufficiency relation. Therefore the combined procedure returns the correct exact verdict on every instance in Γ and runs in polynomial time.

This yields a polynomial-time algorithm for a coNP-complete problem on Γ , hence $\text{coNP} \subseteq P$. Under $P \neq \text{coNP}$, this is impossible. Therefore no solver is simultaneously integrity-preserving and competent on Γ . ■

Corollary 9.12 (Exact Reliability Impossibility in the Hard Regime). *Under the assumptions of the theorem, exact reliability claims are impossible on the full hard regime.*

(L: [DQ28](#), [DQ38](#).)

Corollary 9.13 (Trilemma of Exact Certification). *In the hard regime, no exact certifier can simultaneously be sound, complete on all in-scope instances, and polynomial-budgeted. Operationally, one of three concessions is unavoidable: abstain on some instances, weaken the guarantee, or risk overclaiming.*

(L: [IC30](#), [IC32](#).)

9.4 Witness and Approximation Limits

Hardness does not disappear when one weakens exact minimization to witness checking or approximation. The first theorem shows that even the empty-set core can force exponential witness inspection. The remaining results then show two complementary approximation obstructions: a direct gap obstruction on the shifted hard family and an exact optimization transfer on the set-cover gadget family.

9.4.1 Witness-Checking Lower Bound

Theorem 9.14 (Witness-Checking Lower Bound). *For Boolean decision problems with n coordinates, any sound checker for the empty-set sufficiency core must inspect at least 2^{n-1} witness pairs in the worst case.*

(L: [WD1-3](#).)

Proof. Let $S = \{0, 1\}^n$. Empty-set sufficiency is exactly the claim that Opt is constant on all states. Partition S into 2^{n-1} disjoint witness slots

$$\{(0, z), (1, z)\} \quad \text{for } z \in \{0, 1\}^{n-1}.$$

Each slot can independently carry a disagreement witness (different optimizer values on its two states).

Consider two instance families:

- *YES instance*: Opt constant on all of S .
- *NO_z instance*: identical to YES except slot z has a disagreement.

If a checker inspects fewer than 2^{n-1} slots, at least one slot z^* is uninspected. On all inspected slots, the YES instance and the tailored NO_{z^{*}} instance are identical. The checker therefore sees exactly the same transcript on these two inputs and must return the same answer on both.

But YES is an empty-set-sufficient instance, whereas NO_{z^{*}} is not. So a common answer cannot be sound on both inputs. Hence any sound worst-case checker must inspect every slot, i.e., at least 2^{n-1} witness pairs. ■

9.4.2 Approximation Gap on the Shifted Family

For the shifted reduction family used in the mechanization, the optimum support size exhibits a sharp gap:

$$\text{OPT}(\varphi) = 1 \quad \text{if } \varphi \text{ is a tautology,} \quad \text{OPT}(\varphi) = n + 1 \quad \text{if } \varphi \text{ is not.}$$

One coordinate acts as a gate: toggling it changes the optimizer even before the formula variables are considered, so optimum size is never 0. If φ is a tautology, every branch behind that gate behaves identically, so all non-gate coordinates become irrelevant and the singleton gate coordinate is sufficient. If φ is not a tautology, choose a falsifying assignment a . For each formula coordinate i , compare the open reference state with the state that inserts a at coordinate i : these states agree on all other coordinates but induce different optimal-action sets, so coordinate i is relevant. Together with gate relevance, this forces every coordinate to be present in every sufficient set. That is what creates the exact 1 versus $n + 1$ gap.

Theorem 9.15 (Approximation-hardness theorem on the shifted family). *Fix $\rho \in \mathbb{N}$. Let $\mathcal{A}$ be any solver that, on every shifted-family instance, returns a sufficient set whose cardinality is within factor ρ of optimum. Then for every formula on n coordinates with $\rho < n + 1$,*

$$\varphi \text{ is a tautology} \iff |\mathcal{A}(\varphi)| \leq \rho.$$

(L: [DQ20](#), [DQ39](#), [DQ43](#).)

Proof. **Tautology $\Rightarrow$ threshold passes.** If φ is a tautology, the optimum support size on the shifted family is 1, realized by the singleton gate coordinate. A factor- ρ solver therefore returns a sufficient set of size at most $\rho \cdot 1 = \rho$, so $|\mathcal{A}(\varphi)| \leq \rho$.

Threshold passes $\Rightarrow$ tautology. Assume $|\mathcal{A}(\varphi)| \leq \rho$ with $\rho < n+1$. If φ were not a tautology, then every coordinate would be necessary on the shifted family, so every sufficient set would have size exactly $n+1$. Since $\mathcal{A}(\varphi)$ is sufficient by assumption on $\mathcal{A}$, this would force $|\mathcal{A}(\varphi)| = n+1 > \rho$, contradiction.

Therefore $|\mathcal{A}(\varphi)| \leq \rho$ holds exactly in the tautology case. ■

Theorem 9.16 (Counted-runtime threshold decider). *Fix $\rho \in \mathbb{N}$. Let $\mathcal{A}$ be any counted polynomial-time factor- ρ solver for the shifted minimum-sufficient-set family. Then the derived procedure*

$$\varphi \mapsto \mathbf{1}\{|\mathcal{A}(\varphi)| \leq \rho\}$$

is a counted polynomial-time tautology decider on the gap regime $\rho < n+1$.

(L: [DQ23-24](#).)

Proof. Correctness is exactly the gap-threshold theorem above: on the regime $\rho < n+1$, the predicate $|\mathcal{A}(\varphi)| \leq \rho$ is equivalent to tautology.

For runtime, the derived decider performs two operations: one call to the counted polynomial-time solver $\mathcal{A}$, followed by one cardinality comparison against the fixed threshold ρ . The second step contributes only constant additive overhead relative to the solver run. Hence the derived decision procedure remains counted polynomial-time. ■

9.4.3 Set-Cover Transfer Boundary

Theorem 9.17 (Exact set-cover equivalence on the gadget family). *On the mechanized gadget family, a coordinate set is sufficient if and only if the corresponding set family is a cover. In particular, feasible solutions are in bijection and optimum cardinalities coincide exactly.*

(L: [DQ11](#), [DQ34](#).)

Proof. The gadget is constructed so that each universe element u gives two states, (false, u) and (true, u) , with different optimal-action sets. A coordinate i distinguishes this pair exactly when the corresponding set covers u .

If I is not a cover, choose an uncovered universe element u . Then every coordinate in I takes the same value on (false, u) and (true, u) , but their optimal-action sets differ. So I is not sufficient.

If I is a cover, then for every universe element there exists some coordinate in I that separates the two tagged states attached to that element. Hence two states that agree on all coordinates in I cannot disagree in the tag component in a way that changes the optimizer. The optimizer therefore factors through the I -projection, so I is sufficient.

This yields an exact bijection between feasible sufficient sets and feasible covers, preserving cardinality. Therefore optimum values coincide on the gadget family. ■

Corollary 9.18 (Conditional approximation transfer). *Any factor guarantee or instance-dependent ratio guarantee for minimum sufficiency on the gadget family induces the same guarantee for set cover on that family. Consequently, any impossibility result proved for set cover on that family transfers immediately to minimum sufficiency.*

(L: [DQ25](#), [DQ32-33](#).)

Proof. Compose the candidate minimum-sufficiency solver with the gadget translation and then use the exact equivalence theorem above. Because feasible sets correspond bijectively and preserve cardinality, both approximation factors and instance-dependent ratios are unchanged. ■

The two reductions serve different purposes. The shifted family gives a direct gap obstruction: factor approximation already decides tautology there. The set-cover family gives an exact optimization equivalence: sufficiency is cover there. Together they show that weakening exact minimization to witness checking or approximation does not dissolve the core obstruction.

9.5 Externalized Relevance and Simplicity Tax

The final implication concerns architectural compression. The main point is interpretive rather than algebraic: shrinking a central interface does not remove exact relevance; it relocates the unresolved part of that relevance to some other part of the system. The simplicity tax is the name for that externalized burden.

Definition 9.19 (Relevant Support). For decision problem $\mathcal{D}$, let

$$R(\mathcal{D}) := \{i \in \{1, \dots, n\} : i \text{ is relevant}\}$$

as in Definition 2.8.

Definition 9.20 (Intrinsic and Central Degrees of Freedom). For a model M with native coordinate set $A_M \subseteq \{1, \dots, n\}$:

$$\text{intrinsicDOF}(\mathcal{D}) := |R(\mathcal{D})|, \quad \text{centralDOF}(M, \mathcal{D}) := |R(\mathcal{D}) \cap A_M|.$$

Definition 9.21 (Simplicity Tax).

$$\text{simplicityTax}(M, \mathcal{D}) := |R(\mathcal{D}) \setminus A_M|.$$

Proposition 9.22 (Simplicity-Tax Identity). *For every finite-coordinate pair $(M, \mathcal{D})$,*

$$\text{centralDOF}(M, \mathcal{D}) + \text{simplicityTax}(M, \mathcal{D}) = \text{intrinsicDOF}(\mathcal{D}).$$

(L: [HD5](#), [DQ9](#).)

Proof. Immediate from the partition $R(\mathcal{D}) = (R(\mathcal{D}) \cap A_M) \dot{\cup} (R(\mathcal{D}) \setminus A_M)$. ■

In other words, central simplification changes where exact relevance is handled, not how much exact relevance the deployment must ultimately account for.

Proposition 9.23 (Linear Externalization Under a Per-Site Cost Model). *If each unresolved relevant coordinate induces one unit of per-site external work, then for N decision sites:*

$$\text{ExternalWork}(N) = N \cdot \text{simplicityTax}(M, \mathcal{D}).$$

(L: [HD26](#).)

Proof. Each site pays one unit for each unresolved relevant coordinate, so the total cost is the number of sites times the number of unresolved relevant coordinates. ■

Theorem 9.24 (Amortization Threshold). *Let $H_{\text{central}} > 0$ be one-time centralization cost and $\lambda > 0$ the per-site cost coefficient. If $\text{simplicityTax}(M, \mathcal{D}) > 0$, then for*

$$N > \frac{H_{\text{central}}}{\lambda \cdot \text{simplicityTax}(M, \mathcal{D})},$$

the complete model is cheaper than repeated external handling:

$$H_{\text{central}} < \lambda N \cdot \text{simplicityTax}(M, \mathcal{D}).$$

(L: [HD4](#).)

Proof. Repeated external handling costs $\lambda N \cdot \text{simplicityTax}(M, \mathcal{D})$, while the complete model costs H_{central} once. Solving the strict inequality $H_{\text{central}} < \lambda N \cdot \text{simplicityTax}(M, \mathcal{D})$ for N gives the stated threshold. ■

Corollary 9.25 (Hardness Conservation). *Fix a model M and decision problem $\mathcal{D}$. Coordinates in $R(\mathcal{D}) \setminus A_M$ are still decision-relevant for exact behavior preservation. Hence any exact deployment must handle them somewhere: by enlarging the central model, by performing local resolution, by demanding extra queries, or by abstaining on some cases. In the hard exact regime, Theorem 9.14 and Theorem 9.11 imply that this burden cannot be discharged for free by a polynomial-budget exact certifier.*

(L: [DQ9](#), [HD23](#).)

Proof. Take any coordinate $i \in R(\mathcal{D}) \setminus A_M$. Because i is relevant, there exist witness states whose optimal-action sets differ in a way that cannot be ignored in any exact representation. If a deployment neither models i centrally nor resolves its effect elsewhere, then its exact behavior would factor through the smaller interface A_M , making every coordinate outside A_M irrelevant. That contradicts $i \in R(\mathcal{D})$.

So omitted relevant coordinates must be handled somewhere outside the central interface. In the hard exact regime, the witness lower bound shows that exact certification can require exponentially many witness checks, and the integrity-resource theorem shows that polynomial-budget exact competence is unavailable under $\text{P} \neq \text{coNP}$. Therefore this external burden cannot, in general, be eliminated for free by a polynomial-budget exact certifier. ■

10 Related Work

10.1 Formalized Complexity Theory and Mechanized Reductions

Machine verification of complexity-theoretic arguments remains substantially less mature than formal verification in algebra, analysis, or standard algorithmics. Forster et al. [10] developed certified machine models and computability infrastructure in Coq, and Kunze et al. [19] formalized major proof-theoretic components in Coq as well. In Isabelle/HOL, much of the mature work has centered on verified algorithms and resource analysis for concrete procedures rather than on families of hardness reductions [22, 14]. Lean 4 and Mathlib provide increasingly capable foundations for mechanized finite mathematics and computation [32, 7], but reusable reduction suites for $\text{coNP}/\Sigma_2^P/\text{PP}/\text{PSPACE}$ -style arguments remain relatively rare.

The present artifact is intended as a problem-specific certification layer within that broader program. It does not claim to settle formal complexity theory in full generality; instead, it internalizes the reduction-correctness lemmas, size-bounded hardness packages, exact finite deciders,

counted-search procedures, tractability wrappers, explicit-state upper-bound interfaces, and the scoped witness-over-verifier machinery now used for the stochastic NP^{PP} upper-bound story. In that sense the contribution is closest to a certified reduction-and-decision core for one theorem family rather than to a general-purpose complexity library.

10.2 Rough Sets, Reducts, and Attribute Reduction

The closest classical neighbor is rough-set theory and its large literature on reducts and attribute reduction [24, 18, 17]. That literature asks when a smaller attribute set preserves discernibility or decision-table structure, and it has developed both exact and heuristic methods for reduct computation. There is an obvious family resemblance to the static regime of the present paper: both settings ask which coordinates can be removed without losing decision distinctions.

The difference is not merely terminological. Rough-set reducts are typically formulated relative to indiscernibility relations or decision tables, whereas the object here is the optimizer map of a decision problem and the induced exact preservation of optimal-action sets. That shift matters mathematically as well as computationally. The present paper does not study a single static reduct notion in isolation; it studies one exact certification predicate and then transports it across static, stochastic, and sequential regimes. In the present formulation, insufficiency has short counterexample witnesses, sufficiency is universal over such witnesses, and minimum sufficiency collapses to relevance containment in the static regime. The stochastic and sequential layers then introduce conditional-comparison and policy-preservation phenomena that are outside the usual rough-set formulation. The paper’s static-collapse theorem, regime hierarchy, and exact-certification trilemma are therefore not direct corollaries of the reduct literature, even though that literature is an important intellectual precursor and deserves explicit credit as a nearby tradition.

The same generosity is due to the broader feature-selection and subset-selection literature in machine learning and combinatorial optimization [3, 1]. Those works study sparse predictive representations, informative feature subsets, and the computational cost of choosing them. Our setting differs in targeting exact preservation of the optimal-action correspondence rather than prediction loss, classifier complexity, or approximate empirical utility. Still, the shared concern is clear: identifying what information matters and what can be safely omitted.

10.3 Decision-Theoretic Sufficiency, Informativeness, and Value of Information

There is also a strong conceptual connection to statistical decision theory and the theory of informativeness. Classical sufficient statistics, value-of-information analysis, and comparison of experiments ask when one information structure is at least as useful for decision making as another [2, 30, 26, 15]. That tradition studies when information can be compressed, compared, or discarded without harming decisions. The optimizer quotient and the exact sufficiency predicate in this paper should be read against that background.

At the same time, the present work studies a different layer of the problem. The focus here is not statistical estimation, asymptotic identifiability, or expected utility under a fixed experiment class. It is the computational certification problem of proving that a coordinate projection preserves the optimal-action correspondence exactly. The connection to classical decision theory is therefore one of ancestry and motivation rather than direct theorem transfer. The paper owes that literature recognition for posing, in older language, the same high-level question: what information is genuinely needed for a decision?

10.4 Abstraction, Bisimulation, and Homomorphisms in Planning and Reinforcement Learning

The stochastic and sequential parts of the paper sit near a second major literature: exact abstraction in Markov decision processes, POMDPs, bisimulation-based aggregation, and MDP homomorphisms [23, 20, 21, 12, 27]. This body of work studies when state spaces can be aggregated while preserving value or policy structure, and it has established many of the representation-sensitive complexity phenomena that make richer stochastic and sequential models computationally difficult.

That literature is structurally very close to the present paper and should be credited as such. The main difference is that our predicate is coordinate sufficiency for preserving optimal-action sets under an explicit coordinate-hiding operation. We do not study arbitrary abstract state maps, approximate bisimulation metrics, or value-function approximation guarantees. Instead, we ask for exact certification that a chosen coordinate interface is enough. The contribution here is therefore not abstraction theory in general, but a regime-sensitive complexity theory for one exact decision-relevance predicate tracked coherently across static, stochastic, and sequential settings.

10.5 Explanations, Anchors, and Minimal Decision Rules

The paper should also acknowledge adjacent work in explainable AI and local rule-based explanations, especially anchor-style explanations [29]. Readers will naturally notice the word “anchor” and map it to that literature. The resemblance is real: both settings isolate a restricted condition under which a local piece of information suffices to support a decision or prediction. More broadly, explanation methods based on local rules, counterfactuals, or minimal sufficient subsets share the same informal ambition of identifying a small decision-preserving core.

The difference is again exactness and scope. Anchor explanations in XAI are typically local, model-agnostic, approximate, or precision-calibrated. Our anchor-sufficiency problem is an exact certification problem inside a fully specified decision model, and its complexity is governed by an existential choice followed by a universal fiberwise verification condition. So the paper is not an explanation method, but it is in direct conversation with explanation work that asks what small local structure is enough to preserve a decision.

There is an even closer logical neighbor in recent work on reasons behind decisions, prime-implicant style explanations, and exact decision justifications [6]. This is especially worth noting because it shows that exact small decision-preserving structures are a live contemporary topic, not only a classical one. That literature is concerned with exact structural reasons for a decision outcome and therefore lies much nearer to the present paper than generic post-hoc explanation methods do. The overlap is substantial at the conceptual level: both ask for exact small structures that preserve a decision. The difference is that our objects are coordinate sufficiency, minimum sufficiency, and anchor sufficiency inside coordinate-structured decision problems, and the paper’s main contribution is a regime-sensitive complexity classification of these predicates rather than a logical semantics of reasons alone.

10.6 Abduction, Diagnosis, and Exact Explanatory Cores

The minimum and anchor queries should also be read against the classical literature on abduction and diagnosis [28, 9]. Reiter-style diagnosis studies which hidden fault sets explain an observation, while logic-based abduction studies the complexity of finding explanatory hypotheses sufficient for a target consequence. Those frameworks share an important structural feature with the present work: a move from verification to explanatory search typically raises complexity because one is no longer merely checking a candidate structure but searching for one.

That kinship is especially relevant for the paper’s anchor query. The existential anchor choice is one reason the static anchor problem remains Σ_2^P -complete even when minimum sufficiency collapses in the static regime. In that sense the anchor problem behaves more like explanatory discovery than like plain verification. The exact predicate is different—we certify preservation of optimal-action sets under coordinate hiding rather than explanatory entailment of a formula—but the logical-complexity intuition is nearby enough that this literature deserves explicit acknowledgment.

10.7 Causality, Responsibility, and Structural Explanation

The paper also belongs in conversation with structural accounts of causality, explanation, and responsibility [25, 31, 13, 5]. Those works ask which variables or interventions matter for an outcome, how explanatory structure should be represented, and how responsibility can be assigned to particular factors. This is not the same problem studied here, but it is nearby in spirit: a relevant coordinate is precisely a coordinate whose perturbation can change the decision, and the optimizer quotient isolates exactly the distinctions that matter for optimal-action structure.

The difference, again, is that the present paper focuses on exact decision preservation under coordinate projection and on the computational cost of certifying that preservation. Causality and responsibility frameworks typically emphasize semantic characterization, counterfactual dependence, or intervention-level explanation. The present work instead asks when hidden coordinates can be ignored *without changing the optimizer map*, and how hard it is to certify that claim exactly across static, stochastic, and sequential regimes.

10.8 Oracle, Query, and Access-Based Lower Bounds

Query-access lower bounds provide another nearby technique family [8]. Our witness-checking duality and access-obstruction results belong to that tradition in spirit. The difference is that the lower bounds are developed around the same fixed sufficiency predicate used throughout the paper, which allows direct comparison between forward evaluation, certification, exact minimization, and restricted-access models without changing the underlying decision relation.

10.9 Scope and Novelty

The paper is therefore intentionally synthetic in the good sense. Rough sets and reducts study decision-preserving attribute reduction; feature selection studies informative subsets; statistical decision theory studies informativeness and sufficiency; abstraction and bisimulation literatures study exact aggregation of stochastic and sequential decision processes; explanation methods study local decision-preserving cores; formalized complexity work studies certified reductions and machine-checked decision procedures. The contribution here is to treat exact relevance certification itself as the common object and to develop one coherent complexity-theoretic account around it.

What is new is not merely that these literatures are mentioned together. It is that one predicate—exact preservation of optimal-action distinctions under coordinate hiding—is formalized once and then followed across static, stochastic, and sequential regimes; that the static regime exhibits a non-obvious collapse of minimum sufficiency to relevance containment; that the encoding-sensitive dichotomy isolates a sharp tractable–intractable boundary; that the downstream consequences are pushed through to exact simplification, witness checking, approximation, and the simplicity tax; and that a substantial Lean-backed certification layer is provided for the reduction and finite-decision core. Prior work contains many nearby pieces. The present paper aims to assemble them into a single precise theory of exact relevance certification.

11 Conclusion

This paper develops a regime-sensitive complexity theory of exact relevance certification for coordinate-structured decision problems. Once the state space is factored into coordinates, evaluating the objective on one fully specified state and certifying which coordinates determine the optimal action become distinct computational tasks, and the latter organizes into a sharp hierarchy across static, stochastic, and sequential models.

Main Results

Within the encoding model of Section 2.4, we establish:

- **Static Collapse Theorem plus regime-sensitive complexity matrix:** in the static regime, MINIMUM-SUFFICIENT-SET collapses to relevance containment, so exact minimization is coNP -complete rather than a genuine Σ_2^P search problem, while ANCHOR-SUFFICIENCY remains Σ_2^P -complete. With conditioning, stochastic sufficiency becomes PP -complete and the anchor and minimum variants occupy the existential counting boundary, with PP -hardness and paper-level NPP upper bounds. With temporal dynamics, the sufficiency, anchor, and minimum queries all become PSPACE -complete. Under standard assumptions, these results form a strict hierarchy from static to stochastic to sequential exact certification.
- **Sharp phase transition for exact certification:** logarithmic-size sufficient sets admit polynomial-time algorithms in the explicit regime, while linear-size sufficient sets inherit ETH-conditioned exponential lower bounds in the succinct regime.
- **Structural and downstream consequences:** the paper derives witness and approximation obstructions, exact-simplification hardness transfers, and the trilemma of exact certification: no exact certifier can be simultaneously sound, complete, and polynomial-budgeted on the hard regime. This is a fundamental reliability constraint, and a simplicity-tax framing shows that omitted exact relevance is externalized rather than eliminated.
- **Structured tractability:** bounded actions, separable utility, low tensor rank, tree structure, bounded treewidth, and coordinate symmetry yield polynomial-time certification procedures.

Taken together, these results show that exact relevance certification is a distinct complexity-theoretic object in its own right: computing $\text{Opt}(s)$ on one state can be easy even when proving coordinate irrelevance is hard, and the difficulty of that certification problem depends sharply on whether one is ruling out counterexamples, comparing conditional averages, or preserving policies over time.

Implications

The implications are direct hardness transfers. Configuration simplification is an instance of sufficiency checking, so exact behavior-preserving minimization has no general-purpose polynomial-time worst-case solution unless $\text{P} = \text{coNP}$. Reliable exact certification is likewise regime-limited: under $\text{P} \neq \text{coNP}$, the trilemma of exact certification rules out any solver that is simultaneously sound, complete on the full hard regime, and polynomial-budgeted. And once the canonical relevant-coordinate set $R(\mathcal{D})$ is fixed, any compressed interface merely partitions that relevance into centrally handled and externalized parts. Compression therefore relocates exact relevance rather than erasing it; under the per-site cost model, the unresolved part grows linearly with deployment scale.

This is also the practical meaning of the simplicity tax. In the hard exact regime, a cheap simplification procedure cannot be assumed to have removed decision-relevant structure merely because it has compressed the visible interface. Any relevance omitted from the certified core must still be paid for somewhere else in the system: by local resolution, extra queries, abstention, or weaker guarantees. Exact simplification is therefore costly, but inexact simplification is not free; it externalizes the unresolved burden.

Mechanization

The Lean 4 development certifies the main reduction constructions, size-bounded hardness packages, exact finite deciders, counted finite-search procedures, tractability statements, explicit-state step-counting wrappers, and the scoped witness-over-verifier schema used to support the stochastic NP^{PP} upper bounds. For the stochastic existential anchor family it now also certifies the existential-majority reduction core and an internal completeness-style surrogate (*L*: [EH2-5](#).) This gives a mechanically checked core for the reduction and finite-decision layers of the argument. Standard PP/PSPACE and oracle-class membership arguments remain stated in the paper rather than packaged as end-to-end Turing-machine witness proofs.

Outlook

Two immediate directions remain. First, the reduction infrastructure can be integrated more tightly with general-purpose formalized complexity libraries. Second, the artifact boundary can be made even cleaner by continuing to package the existing finite-decision results into more uniform summary interfaces without changing the paper’s complexity claims. On the complexity side, the remaining gap is now sharply localized: the stochastic anchor and minimum variants have a complete paper-level placement, but a fully standard oracle-machine formalization of the NP^{PP} boundary remains outside the current repository, and whether the existential layer collapses further to PP under the present encoding remains unclear.

Acknowledgments

Generative AI tools (including Codex, Claude Code, Augment, Kilo, and OpenCode) were used throughout this manuscript, across all sections (Abstract, Introduction, theoretical development, proof sketches, applications, conclusion, and appendix) and across all stages from initial drafting to final revision. The tools were used for boilerplate generation, prose and notation refinement, $\text{\LaTeX}$ /structure cleanup, translation of informal proof ideas into candidate formal artifacts (Lean/ $\text{\LaTeX}$), and repeated adversarial reviewer-style critique passes to identify blind spots and clarity gaps.

The author retained full intellectual and editorial control, including problem selection, theorem statements, assumptions, novelty framing, acceptance criteria, and final inclusion/exclusion decisions. No technical claim was accepted solely from AI output. Formal claims reported as machine-verified were admitted only after Lean verification (no `sorry` in cited modules) and direct author review; Lean was used as an integrity gate for responsible AI-assisted research. The author is solely responsible for all statements, citations, and conclusions.

A Lean Handle Index

The table below lists the auto-generated Lean handle identifiers cited inline in the paper.

ID	Handle	ID	Handle
AB2	DecisionProblem.surjective_abstraction_factors_or_erases DecisionQuotient/AbstractionCollapse.lean	CC1	ClaimClosure.DQ8 DecisionQuotient/ClaimClosure.lean
CR1	DecisionQuotient.ConfigReduction.config_sufficiency_iff_ behavior_preserving DecisionQuotient/Hardness/ConfigReduction.lean	CT12	DecisionQuotient.Physics.ClaimTransport.physical_bridge_ bundle DecisionQuotient/Physics/ClaimTransport.lean
DC19	StochasticSequential.stochastic_anchor_sufficient_of_ stochastic_sufficient DecisionQuotient/StochasticSequential/Quotient.lean	DC23	StochasticSequential.sequential_anchor_sufficient_of_ sequential_sufficient DecisionQuotient/StochasticSequential/Basic.lean
DC28	StochasticSequential.reduceTQBF_correct_anchor DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC29	StochasticSequential.reduceTQBF_to_sequential_anchor_ reduction DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC40	StochasticSequential.reduceMAJSATPureAnchor_correct DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC41	StochasticSequential.reduceMAJSAT_to_pure_stochastic_ anchor_reduction DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC42	StochasticSequential.stochastic_anchor_check_pp_hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC44	StochasticSequential.sequential_anchor_check_pspace_ hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC45	StochasticSequential.stochastic_sufficiency_pp_hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC46	StochasticSequential.sequential_sufficiency_pspace_hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC47	StochasticSequential.stochastic_minimum_sufficiency_pp_ hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean	DC48	StochasticSequential.sequential_minimum_sufficiency_ pspace_hard DecisionQuotient/StochasticSequential/PolynomialReduction.lean
DC49	StochasticSequential.sequentialMinimalSufficient_iff_ relevant DecisionQuotient/StochasticSequential/Basic.lean	DC50	StochasticSequential.sequentialRelevantSet_is_minimal DecisionQuotient/StochasticSequential/Basic.lean
DC57	StochasticSequential.fiberDecisionProblem_sufficient DecisionQuotient/StochasticSequential/Basic.lean	DC62	StochasticSequential.countedStochasticSufficiencySearch_ spec DecisionQuotient/StochasticSequential/Computation.lean
DC63	StochasticSequential.countedStochasticSufficiencySearch_ steps DecisionQuotient/StochasticSequential/Computation.lean	DC64	StochasticSequential.countedStochasticAnchorSearch_spec DecisionQuotient/StochasticSequential/Computation.lean
DC65	StochasticSequential.countedStochasticAnchorSearch_steps DecisionQuotient/StochasticSequential/Computation.lean	DC66	StochasticSequential.countedSequentialSufficiencySearch_ spec DecisionQuotient/StochasticSequential/Computation.lean
DC67	StochasticSequential.countedSequentialSufficiencySearch_ steps DecisionQuotient/StochasticSequential/Computation.lean	DC68	StochasticSequential.countedSequentialAnchorSearch_spec DecisionQuotient/StochasticSequential/Computation.lean
DC69	StochasticSequential.countedSequentialAnchorSearch_steps DecisionQuotient/StochasticSequential/Computation.lean	DC70	DecisionQuotient.static_sufficiency_inP_explicit DecisionQuotient/ExplicitStateMembership.lean
DC71	DecisionQuotient.static_anchor_inP_explicit DecisionQuotient/ExplicitStateMembership.lean	DC72	StochasticSequential.stochastic_sufficiency_inP_explicit DecisionQuotient/StochasticSequential/Computation.lean
DC73	StochasticSequential.stochastic_anchor_inP_explicit DecisionQuotient/StochasticSequential/Computation.lean	DC74	StochasticSequential.sequential_sufficiency_inP_explicit DecisionQuotient/StochasticSequential/Computation.lean
DC75	StochasticSequential.sequential_anchor_inP_explicit DecisionQuotient/StochasticSequential/Computation.lean	DC76	DecisionQuotient.explicit_state_inP_summary DecisionQuotient/ExplicitStateMembership.lean
DC80	DecisionQuotient.staticSufficiency_counted_search_witness DecisionQuotient/AlgorithmComplexity.lean	DC81	DecisionQuotient.static_query_search_matrix DecisionQuotient/AlgorithmComplexity.lean
DC82	DecisionQuotient.finite_search_summary DecisionQuotient/ComplexityMain.lean	DP1	DecisionQuotient.DecisionProblem.minimalSufficient_iff_ relevant
DP2	DecisionQuotient.DecisionProblem.relevantSet_is_minimal	DP6	ClaimClosure.DP6 DecisionQuotient/ClaimClosure.lean
DP7	ClaimClosure.DP7 DecisionQuotient/ClaimClosure.lean	DQ1	ClaimClosure.DQ1 DecisionQuotient/ClaimClosure.lean
DQ2	ClaimClosure.DQ2 DecisionQuotient/ClaimClosure.lean	DQ3	ClaimClosure.DQ3 DecisionQuotient/ClaimClosure.lean
DQ6	ClaimClosure.DQ6 DecisionQuotient/ClaimClosure.lean	DQ7	ClaimClosure.DQ7 DecisionQuotient/ClaimClosure.lean
DQ9	DecisionQuotient.HardnessDistribution.simplicityTax_ conservation DecisionQuotient/HardnessDistribution.lean	DQ10	DecisionQuotient.IntegrityCompetence.alwaysAbstain_ integrity DecisionQuotient/IntegrityCompetence.lean
DQ11	DecisionQuotient.SetCoverInstance.min_sufficient_iff_set_ cover DecisionQuotient/Hardness/SetCoverReduction.lean	DQ12	DecisionQuotient.StochasticSequential.bounded_horizon_ tractable DecisionQuotient/StochasticSequential/Tractability.lean
DQ13	DecisionQuotient.StochasticSequential.bounded_support_ tractable DecisionQuotient/StochasticSequential/Tractability.lean	DQ14	DecisionQuotient.StochasticSequential.fully_observable_ tractable DecisionQuotient/StochasticSequential/Tractability.lean
DQ15	DecisionQuotient.StochasticSequential.product_ distribution_tractable DecisionQuotient/StochasticSequential/Tractability.lean	DQ16	DecisionQuotient.StochasticSequential.product_enables_ transfer DecisionQuotient/StochasticSequential/CrossRegime.lean
DQ18	DecisionQuotient.StochasticSequential.static_simpler_ than_stochastic DecisionQuotient/StochasticSequential/CrossRegime.lean	DQ19	DecisionQuotient.StochasticSequential.stochastic_simpler_ than_sequential DecisionQuotient/StochasticSequential/CrossRegime.lean
DQ20	DecisionQuotient.all_coordinates_necessary_of_not_ tautology DecisionQuotient/Hardness/MinSufficientApproximation.lean	DQ25	DecisionQuotient.counted_min_sufficient_ inapproximability_conditional DecisionQuotient/Hardness/ApproximationHardness.lean
DQ26	DecisionQuotient.dichotomy_conditional DecisionQuotient/ClaimClosure.lean	DQ27	DecisionQuotient.eth_lower_bound_informal DecisionQuotient/Hardness/ETH.lean
DQ28	DecisionQuotient.exact_certainty_inflation_under_ hardness_core DecisionQuotient/ClaimClosure.lean	DQ30	DecisionQuotient.hard_family_all_coords_core DecisionQuotient/ClaimClosure.lean
DQ31	DecisionQuotient.integrity_resource_bound_for_sufficiency DecisionQuotient/ClaimClosure.lean	DQ32	DecisionQuotient.min_sufficient_factor_inapprox_of_set_ cover_factor_inapprox DecisionQuotient/Hardness/ApproximationHardness.lean
DQ34	DecisionQuotient.min_sufficient_set_cover_equiv DecisionQuotient/Hardness/ApproximationHardness.lean	DQ36	DecisionQuotient.minsuff_conp_complete_conditional DecisionQuotient/ClaimClosure.lean
DQ38	DecisionQuotient.no_exact_claim_admissible_under_ hardness_core DecisionQuotient/ClaimClosure.lean	DQ39	DecisionQuotient.singleton_gate_sufficient_of_tautology DecisionQuotient/Hardness/MinSufficientApproximation.lean
DQ40	DecisionQuotient.sufficiency_conp_complete_conditional DecisionQuotient/ClaimClosure.lean	DQ42	DecisionQuotient.sufficient_means_factorizable DecisionQuotient/Information.lean
DQ43	DecisionQuotient.tautology_decidable_from_factor_approx DecisionQuotient/Hardness/MinSufficientApproximation.lean	DQ44	DecisionQuotient.tractable_subcases_conditional DecisionQuotient/ClaimClosure.lean
DQ45	DecisionQuotient.tractable_tree_core DecisionQuotient/ClaimClosure.lean	DQ51	DecisionQuotient.bounded_actions_tractable DecisionQuotient/Summary.lean
DQ56	DecisionQuotient.orbitType_count_bound DecisionQuotient/Tractability/Dimensional.lean	DQ57	DecisionQuotient.separable_utility_tractable DecisionQuotient/Summary.lean
DQ58	DecisionQuotient.stochastic_objective_bridge_can_fail_ on_sufficiency DecisionQuotient/ClaimClosure.lean	DQ62	DecisionQuotient.transition_coupled_bridge_can_fail_on_ sufficiency DecisionQuotient/ClaimClosure.lean
DQ63	DecisionQuotient.tree_structure_tractable DecisionQuotient/Summary.lean	DQ64	DecisionQuotient.SeparableUtility DecisionQuotient/Tractability/SeparableUtility.lean
DQ72	DecisionQuotient.bounded_actions_complexity DecisionQuotient/Tractability/BoundedActions.lean	DQ74	DecisionQuotient.sufficiency_poly_separable DecisionQuotient/Tractability/SeparableUtility.lean

ID	Handle	ID	Handle
DQ75	DecisionQuotient.tensor_contraction_tractable DecisionQuotient/Tractability/SeparableUtility.lean	DQ76	DecisionQuotient.low_rank_utility_admits_factored_computation DecisionQuotient/Tractability/SeparableUtility.lean
DQ77	DecisionQuotient.low_rank_tractability DecisionQuotient/Tractability/SeparableUtility.lean	DQ78	DecisionQuotient.sufficiency_poly_tree_structured DecisionQuotient/Tractability/TreeStructure.lean
DQ79	DecisionQuotient.csp_treewidth_tractable DecisionQuotient/Tractability/TreeStructure.lean	DQ80	DecisionQuotient.sufficiency_reduces_to_interaction_csp DecisionQuotient/Tractability/TreeStructure.lean
DQ81	DecisionQuotient.bounded_treewidth_tractability DecisionQuotient/Tractability/TreeStructure.lean	DQ82	DecisionQuotient.orbitType_eq_iff DecisionQuotient/Tractability/Dimensional.lean
DQ83	DecisionQuotient.symmetric_optimalActions_orbit_invariant DecisionQuotient/Tractability/Dimensional.lean	DQ84	DecisionQuotient.sufficiency_reduces_to_cross_orbit_check DecisionQuotient/Tractability/Dimensional.lean
DQ85	DecisionQuotient.symmetric_sufficiency_complexity_bound DecisionQuotient/Tractability/Dimensional.lean	EH1	StochasticSequential.existential_anchor_source_fits_np_over_ppstyle_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean
EH2	StochasticSequential.existential_anchor_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean	EH3	StochasticSequential.existential_anchor_query_family_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean
EH4	StochasticSequential.existential_anchor_np_over_ppstyle_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean	EH5	StochasticSequential.existential_anchor_query_family_np_over_ppstyle_hard_honest DecisionQuotient/StochasticSequential/ExistentialHardness.lean
HD4	DecisionQuotient.HardnessDistribution.complete_model_dominates_after_threshold DecisionQuotient/HardnessDistribution.lean	HD5	DecisionQuotient.HardnessDistribution.gap_conservation_card DecisionQuotient/HardnessDistribution.lean
HD14	DecisionQuotient.HardnessDistribution.linear_lt_exponential_plus_constant_eventually DecisionQuotient/HardnessDistribution.lean	HD23	DecisionQuotient.HardnessDistribution.hardness_is_irreducible_required_work DecisionQuotient/HardnessDistribution.lean
HD26	DecisionQuotient.HardnessDistribution.totalExternalWork_eq_n_mul_gapCard DecisionQuotient/HardnessDistribution.lean	IC30	DecisionQuotient.IntegrityCompetence.exactCertaintyInflation_iff_no_exact_competence DecisionQuotient/IntegrityCompetence.lean
IC32	DecisionQuotient.IntegrityCompetence.integrity_forces_abstention DecisionQuotient/IntegrityCompetence.lean	IC33	DecisionQuotient.IntegrityCompetence.integrity_not_competent_of_nonempty_scope DecisionQuotient/IntegrityCompetence.lean
IC34	DecisionQuotient.IntegrityCompetence.integrity_resource_bound DecisionQuotient/IntegrityCompetence.lean	OU1	StochasticSequential.stochastic_anchor_query_fits_np_over_ppstyle DecisionQuotient/StochasticSequential/OracleUpperBounds.lean
OU2	StochasticSequential.stochastic_minimum_query_fits_np_over_ppstyle DecisionQuotient/StochasticSequential/OracleUpperBounds.lean	OU6	StochasticSequential.stochastic_anchor_inP_explicit_via_witness_schema DecisionQuotient/StochasticSequential/OracleUpperBounds.lean
OU7	StochasticSequential.stochastic_minimum_inP_explicit_via_witness_schema DecisionQuotient/StochasticSequential/OracleUpperBounds.lean	PA3	Physics.AnchorChecks.stochastic_anchor_check_iff_exists_anchor_singleton DecisionQuotient/Physics/AnchorChecks.lean
QT1	DecisionProblem.quotient_is_coarsest DecisionQuotient/Quotient.lean	QT7	DecisionProblem.quotient_has_unique_factorization DecisionQuotient/Quotient.lean
WD1	DecisionQuotient.checking_witnessing_duality_budget DecisionQuotient/WitnessCheckingDuality.lean	WD2	DecisionQuotient.no_sound_checker_below_witness_budget DecisionQuotient/WitnessCheckingDuality.lean
WD3	DecisionQuotient.checking_time_ge_witness_budget DecisionQuotient/WitnessCheckingDuality.lean		

References

- [1] Edoardo Amaldi and Viggo Kann. On the approximability of minimizing nonzero variables or unsatisfied relations in linear systems. *Theoretical Computer Science*, 209(1-2):237–260, 1998.
- [2] David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2):265–272, 1953.
- [3] Avrim L. Blum and Pat Langley. Selection of relevant features and examples in machine learning. *Artificial Intelligence*, 97(1-2):245–271, 1997.
- [4] Hans L. Bodlaender. A tourist guide through treewidth. In *Acta Cybernetica*, volume 11, pages 1–21, 1993.
- [5] Hana Chockler and Joseph Y. Halpern. Responsibility and blame: A structural-model approach. *Journal of Artificial Intelligence Research*, 22:93–115, 2004.
- [6] Adnan Darwiche and Auguste Hirth. On the (complete) reasons behind decisions. *Journal of Logic, Language and Information*, 32(1):63–88, 2023.
- [7] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pages 625–635. Springer, 2021.
- [8] Shahar Dobzinski and Jan Vondrák. From query complexity to computational complexity. In *Proceedings of the 44th ACM Symposium on Theory of Computing*, pages 1107–1116, 2012.

- [9] Thomas Eiter and Georg Gottlob. The complexity of logic-based abduction. *Journal of the ACM*, 42(1):3–42, 1995.
- [10] Yannick Forster, Edith Heiter, and Gert Smolka. Verified programming of Turing machines in Coq. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 114–127. ACM, 2019.
- [11] Eugene C. Freuder. Complexity of k-tree structured constraint satisfaction problems. *Proceedings of AAAI*, pages 4–9, 1990.
- [12] Robert Givan, Thomas Dean, and Matthew Greig. Equivalence notions and model minimization in markov decision processes. *Artificial Intelligence*, 147(1-2):163–223, 2003.
- [13] Joseph Y. Halpern and Judea Pearl. Causes and explanations: A structural-model approach. part ii: Explanations. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence*, pages 27–34, 2001.
- [14] Maximilian P. L. Haslbeck and Tobias Nipkow. Verified analysis of list update algorithms. In *Journal of Automated Reasoning*, volume 65, pages 117–156. Springer, 2021.
- [15] Ronald A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966.
- [16] Russell Impagliazzo and Ramamohan Paturi. On the complexity of k -sat. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.
- [17] Richard Jensen. Rough set-based feature selection: A review. In *Rough Computing: Theories, Technologies and Applications*, pages 91–129. IGI Global, 2008.
- [18] Richard Jensen and Qiang Shen. Semantics-preserving dimensionality reduction: Rough and fuzzy-rough-based approaches. *IEEE Transactions on Knowledge and Data Engineering*, 16(12):1457–1471, 2004.
- [19] Fabian Kunze, Gert Smolka, and Yannick Forster. Formal small-step verification of a call-by-value lambda calculus machine. In *Asian Symposium on Programming Languages and Systems*, pages 264–283. Springer, 2019.
- [20] Michael L. Littman, Judy Goldsmith, and Martin Mundhenk. The computational complexity of probabilistic planning. *Journal of Artificial Intelligence Research*, 9:1–36, 1998.
- [21] Martin Mundhenk, Judy Goldsmith, Christopher Lusena, and Eric Allender. The complexity of finite-horizon markov decision process problems. *Journal of the ACM*, 47(4):681–720, 2000.
- [22] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer, 2002.
- [23] Christos H. Papadimitriou and John N. Tsitsiklis. The complexity of markov decision processes. *Mathematics of Operations Research*, 12(3):441–450, 1987.
- [24] Zdzisław Pawlak. Rough sets. *International Journal of Parallel Programming*, 11(5):341–356, 1982.
- [25] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

- [26] Howard Raiffa and Robert Schlaifer. *Applied Statistical Decision Theory*. Harvard University Press, 1961.
- [27] Balaraman Ravindran. *An Algebraic Approach to Abstraction in Reinforcement Learning*. PhD thesis, University of Massachusetts Amherst, 2004.
- [28] Raymond Reiter. A theory of diagnosis from first principles. *Artificial Intelligence*, 32(1):57–95, 1987.
- [29] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Anchors: High-precision model-agnostic explanations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [30] Leonard J. Savage. *The Foundations of Statistics*. John Wiley & Sons, 1954.
- [31] Peter Spirtes, Clark Glymour, and Richard Scheines. *Causation, Prediction, and Search*. MIT Press, 2nd edition, 2000.
- [32] The mathlib Community. The Lean mathematical library. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020.